

**VIETNAM NATIONAL UNIVERSITY, HANOI**  
**UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



**Bui Minh Thang**

**DESIGN AND EVALUATION OF A CONTEXT-AWARE  
RESOURCE PROFILE SELECTION EXTENSION FOR  
JUPYTERHUB ON KUBERNETES**

**GRADUATION THESIS**

**Major: Computer Networks and Data Communications**

**HA NOI – 2026**

**VIETNAM NATIONAL UNIVERSITY, HANOI**  
**UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

**Bui Minh Thang**

**DESIGN AND EVALUATION OF A CONTEXT-AWARE  
RESOURCE PROFILE SELECTION EXTENSION FOR  
JUPYTERHUB ON KUBERNETES**

**GRADUATION THESIS**

**Major: Computer Networks and Data Communications**

**Supervisor: Dr. Hoang Xuan Tung**

**HA NOI – 2026**

## ABSTRACT

JupyterHub provides a flexible platform for interactive computing, but selecting an appropriate computational environment remains a manual configuration task. Users are typically required to understand available resource profiles, software environments, and notebook images before starting a session. As the number and diversity of available environments increase, manual selection can introduce unnecessary configuration effort and may result in unsuitable resource or software choices. This thesis investigates the use of workload intent and context to assist users in selecting an appropriate JupyterHub environment while preserving administrator-defined resource and deployment policies.

The problem addressed in this thesis is the selection of a suitable resource profile and notebook environment from a bounded, administrator-managed catalog based on a user’s natural-language workload description. The objective is not to allow an automated model to generate arbitrary Kubernetes configurations, but to determine whether natural-language intent can be transformed into a structured representation, used to retrieve relevant approved environments, and subsequently filtered and ranked according to explicit technical constraints. A further objective is to establish a system architecture in which recommendation remains explainable, policy-bounded, reproducible, and subject to user confirmation before deployment.

The proposed approach, implemented as Intent-Spawner, combines structured intent extraction, hybrid information retrieval, deterministic constraint evaluation, and trusted catalog resolution. A user’s workload description is converted into a schema-validated structured intent containing task characteristics, software requirements, resource constraints, and preferences. Candidate environments are then retrieved using two complementary retrieval channels: BM25-based sparse retrieval for exact technical terminology and dense semantic retrieval for semantically related workload descriptions. Their ranked results are combined using Reciprocal Rank Fusion (RRF). Retrieved candidates are subsequently evaluated against hard constraints, including GPU availability, minimum CPU and memory requirements, dataset size, and forbidden features. Candidates that violate hard constraints are excluded before deterministic preference ranking is applied.

The resulting recommendation is resolved through an administrator-owned candidate catalog rather than allowing the recommendation model to directly specify an image or Kubernetes

resource configuration. Approved notebook images are identified by immutable digest-pinned references, while resource profiles are restricted to administrator-defined configurations. A preview and confirmation mechanism binds the displayed recommendation to the authenticated user and to a single-use, time-limited token. The JupyterHub pre-spawn hook validates this state before applying the selected configuration, without performing new recommendation or model inference during pod creation. The system was evaluated through controlled experiment protocols covering recommendation behavior, system integration, resource-related measurements, and environment functionality. The evaluation distinguishes workload families as the semantic experimental unit and separates repeated executions used for stability analysis from independent recommendation cases.

The experimental evaluation demonstrates that the implemented recommendation pipeline can process structured workload intents and resolve recommendations through the bounded candidate catalog without requiring arbitrary configuration generation. In the evaluated external-model extraction runs, the structured-intent stage produced valid outputs for all 240 evaluated cases, while the corresponding evaluation pipeline completed without recommendation fallbacks in the 320-trial workload evaluation. The evaluation also established the functional integration of recommendation preview, policy validation, user confirmation, and pre-spawn application within JupyterHub, including enforcement of approved profiles and digest-pinned images.

These results indicate that intent-aware environment selection can be integrated into JupyterHub while maintaining a separation between language-based recommendation and authoritative infrastructure configuration. Hybrid retrieval provides a mechanism for combining exact technical vocabulary with semantic similarity, whereas deterministic constraints prevent soft relevance signals from overriding explicit resource or policy requirements. The architecture therefore provides a bounded form of decision support in which natural-language interaction reduces configuration effort without transferring deployment authority to the recommendation model. However, the available evidence does not establish statistical superiority over manual selection or alternative recommenders, nor does it demonstrate general performance across arbitrary multi-node, multi-tenant Kubernetes deployments. Human-user effectiveness and broader production validation remain subjects for further evaluation.

**Keywords:** JupyterHub, Kubernetes, Workload Intent, Resource Profile Recommendation, Hybrid Information Retrieval, Constraint-Based Selection, Notebook Environments.

## **AUTHORSHIP**

I hereby declare that the work contained in this thesis is of my own and has not been previously submitted for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no materials previously published or written by another person except where due reference or acknowledgement is made.

Signature: \_\_\_\_\_

## **ACKNOWLEDGEMENTS**

I would like to express my sincere gratitude to my thesis supervisor, Dr. Hoang Xuan Tung, for guidance, critical feedback, and encouragement throughout the development and evaluation of this work. I am also grateful to the lecturers and staff of the Faculty of Information Technology, for the technical and academic foundation that made this project possible.

I would also like to thank my friends, and family for their valuable discussions, encouragement, and practical support throughout this iterative process. I take full responsibility for any remaining errors, limitations, or interpretation choices in this thesis.

# Contents

|                                                           |      |
|-----------------------------------------------------------|------|
| <b>Abstract</b> .....                                     | i    |
| <b>Authorship</b> .....                                   | ii   |
| <b>Acknowledgements</b> .....                             | iii  |
| <b>List of Figures</b> .....                              | viii |
| <b>List of Tables</b> .....                               | x    |
| <b>List of Algorithms</b> .....                           | xi   |
| <b>Abbreviations</b> .....                                | xii  |
| <b>Chapter 1. Introduction</b> .....                      | 1    |
| 1.1. Problem statement .....                              | 1    |
| 1.2. Contributions and thesis overview .....              | 3    |
| <b>Chapter 2. Background</b> .....                        | 5    |
| 2.1. JupyterHub .....                                     | 5    |
| 2.2. Kubernetes-based notebook execution .....            | 8    |
| 2.2.1. Containers and pods .....                          | 8    |
| 2.2.2. CPU and memory resources .....                     | 9    |
| 2.2.3. Resource profiles .....                            | 10   |
| 2.3. Notebook container images .....                      | 11   |
| 2.4. Workload intent .....                                | 12   |
| 2.5. Information retrieval .....                          | 13   |
| 2.5.1. Sparse retrieval and BM25 .....                    | 14   |
| 2.5.2. Dense retrieval .....                              | 15   |
| 2.5.3. Hybrid retrieval with reciprocal rank fusion ..... | 16   |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 2.6. Constraint-based recommendation .....                          | 17        |
| <b>Chapter 3. Proposed Method .....</b>                             | <b>19</b> |
| 3.1. Overview .....                                                 | 19        |
| 3.2. Structured intent representation .....                         | 21        |
| 3.3. Hybrid candidate retrieval .....                               | 24        |
| 3.3.1. Dual-channel candidate retrieval pipeline .....              | 24        |
| 3.3.2. Sparse lexical query construction and execution.....         | 25        |
| 3.3.3. Dense semantic indexing and query inference .....            | 26        |
| 3.3.4. Reciprocal rank fusion implementation .....                  | 26        |
| 3.4. Deterministic constraint evaluation .....                      | 27        |
| 3.5. Deterministic candidate ranking .....                          | 31        |
| 3.5.1. Multi-Criteria deterministic scoring formulation .....       | 31        |
| 3.5.2. Linear rank utility and resource cost modeling.....          | 31        |
| 3.6. JupyterHub integration and runtime enforcement .....           | 33        |
| 3.6.1. Trusted candidate resolution and policy validation .....     | 33        |
| 3.6.2. Two-phase preview and confirmation lifecycle .....           | 34        |
| 3.6.3. Pre-spawn binding hook and zero-recomputation execution..... | 35        |
| <b>Chapter 4. Experiments and evaluation .....</b>                  | <b>37</b> |
| 4.1. Experimental design .....                                      | 37        |
| 4.1.1. Experiment streams and evidence organization .....           | 38        |
| 4.1.2. Evaluation environment and reproducibility .....             | 39        |
| 4.2. Offline recommendation and robustness evaluation .....         | 40        |
| 4.2.1. Stream E1 .....                                              | 40        |
| 4.2.2. Candidate retrieval channel ablation.....                    | 43        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| 4.2.3. Stream E2 .....                                             | 45        |
| 4.2.4. Cross-metric and trade-off analysis .....                   | 46        |
| 4.3. Kubernetes cluster resource evaluation .....                  | 47        |
| 4.3.1. Stream E4 .....                                             | 47        |
| 4.3.2. In-container resource sampling and envelope profiling ..... | 48        |
| 4.4. Container image capability and storage accounting .....       | 50        |
| 4.4.1. Stream E5: container image capability probing .....         | 50        |
| 4.4.2. Stream E5: compressed OCI layer storage accounting .....    | 52        |
| 4.5. Human interaction evaluation protocol.....                    | 56        |
| 4.5.1. Stream E3 .....                                             | 56        |
| 4.5.2. Empirical status and reporting transparency .....           | 57        |
| <b>Chapter 5. Conclusion</b> .....                                 | <b>59</b> |
| 5.1. Conclusion .....                                              | 59        |
| 5.2. Future work .....                                             | 60        |
| <b>References</b> .....                                            | <b>61</b> |

## List of Figures

|                                                                                                                                                                                                                           |    |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [9] .....                                                                                   | 7  |
| 2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [15] .....                                                                                                              | 9  |
| 2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [4] .....                                                                                                    | 11 |
| 3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture .....                                                                                                                             | 20 |
| 3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion .....                                                                                           | 25 |
| 3.3. Deterministic hard constraint evaluation decision tree and pruning logic .....                                                                                                                                       | 29 |
| 3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle .....                                                                                                                                          | 34 |
| 4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting ..... | 38 |
| 4.2. Offline recommendation quality and robustness metrics in Stream E1, comparing P1 (rule-based) and P2 (Intent-Spawner) with macro-averaged point estimates and 95% percentile bootstrap confidence intervals.....     | 42 |
| 4.3. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion. ....                                                  | 44 |
| 4.4. Observed Kubernetes Pod execution reliability across 421 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right). ....      | 48 |

|                                                                                                                                                                                                                                     |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.5. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment. ....                                       | 49 |
| 4.6. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C)..... | 51 |
| 4.7. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers. ....                                               | 54 |
| 4.8. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images. ....                                               | 55 |
| 4.9. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments. ....                                     | 56 |

## List of Tables

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| 3.1. Specification of the StructuredIntent data contract and normalization rules .....           | 22 |
| 3.2. Deterministic hard constraint evaluation rules and pruning conditions .....                 | 28 |
| 4.1. ....                                                                                        | 39 |
| 4.2 Experiment streams, evaluated constructs, and empirical evidence scope .....                 | 39 |
| 4.3. ....                                                                                        | 41 |
| 4.4 Observed E1 offline recommendation quality estimates for P1 and P2 .....                     | 41 |
| 4.5. ....                                                                                        | 45 |
| 4.6 Observed E2 semantic robustness metrics on the development split .....                       | 45 |
| 4.7. ....                                                                                        | 47 |
| 4.8 Observed E4 Kubernetes Pod execution outcomes (OrbStack environment) .....                   | 47 |
| 4.9. ....                                                                                        | 52 |
| 4.10 E5 container image functional capability probe evaluation results .....                     | 52 |
| 4.11. ....                                                                                       | 53 |
| 4.12 Observed compressed OCI layer-byte accounting and deduplication across four<br>images ..... | 53 |
| 4.13. ....                                                                                       | 54 |
| 4.14 Marginal unique compressed layer bytes introduced by image order .....                      | 54 |

## List of Algorithms

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| 3.1. Structured intent extraction, normalization, and validation procedure .....        | 23 |
| 3.2. Reciprocal Rank Fusion for hybrid candidate retrieval .....                        | 27 |
| 3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking ..... | 33 |
| 3.4. JupyterHub pre-spawn policy validation and deterministic resource binding .....    | 36 |

## Abbreviations

|      |                                                     |
|------|-----------------------------------------------------|
| API  | Application Programming Interface                   |
| BM25 | Best Matching 25                                    |
| CPU  | Central Processing Unit                             |
| GPU  | Graphics Processing Unit                            |
| HCI  | Human-Computer Interaction                          |
| IR   | Information Retrieval                               |
| LLM  | Large Language Model                                |
| MRR  | Mean Reciprocal Rank                                |
| nDCG | Normalized Discounted Cumulative Gain               |
| OCI  | Open Container Initiative                           |
| POD  | Plain Old Data / Kubernetes Pod (context dependent) |
| RRF  | Reciprocal Rank Fusion                              |
| RQ   | Research Question                                   |
| TTL  | Time to Live                                        |
| VPA  | Vertical Pod Autoscaler                             |

# Chapter 1.

## Introduction

### 1.1. Problem statement

The development of interactive computing and container orchestration has progressively changed how computational environments are provided to users. In 2014, Project Jupyter emerged from the IPython project with a broader focus on interactive data science and scientific computing across programming languages. Around the same period, Kubernetes was introduced as an open-source system for managing containerized workloads, with its first public commit in June 2014 and its 1.0 release in July 2015. JupyterHub subsequently emerged as a multi-user platform for providing managed computational environments, with its first preview release in March 2015. The combination of JupyterHub and container orchestration technologies such as Kubernetes has since provided a practical foundation for delivering isolated and centrally managed notebook environments to groups of users. Today, JupyterHub explicitly supports Kubernetes-based deployments and is designed to provide users with computational environments and resources while allowing administrators to manage the underlying infrastructure.

Interactive computational environments have become an important part of modern data analysis, scientific computing, education, and machine learning. Jupyter notebooks are particularly useful in these settings because they allow users to combine executable code, computational results, and explanatory text within a single interactive environment. In multi-user deployments, however, providing notebook environments is not only a matter of running the notebook application itself. The platform must also manage users, compute resources, storage, software environments, and isolation between workloads. JupyterHub provides a common platform for deploying and managing multi-user Jupyter environments,

while Kubernetes can provide the underlying infrastructure for scalable and isolated execution. In a typical JupyterHub deployment on Kubernetes, KubeSpawner creates a Kubernetes Pod for each user’s notebook server and can expose a set of administrator-defined resource profiles and container images. Such profiles allow administrators to control the available CPU, memory, accelerators, and software environments while preventing users from directly requesting arbitrary infrastructure configurations. This administrator-controlled approach provides an important operational boundary, but it also introduces a usability problem. Users generally describe their requirements in terms of the work they want to perform rather than in terms of infrastructure configuration. For example, a user may know that they need to train a machine-learning model on a large dataset, process images, or perform data analysis with a particular framework. The platform, however, may require the user to select an appropriate resource profile and notebook image from a predefined catalog. Consequently, users may need to understand technical differences between available environments before they can start their actual work. This creates a gap between workload intent and infrastructure configuration. A user understands the intended computation, while the platform requires a concrete profile and image selection. Bridging this gap is particularly relevant when a catalog contains multiple environments with different resource capacities and software capabilities.

The conventional solution is to present users with a list of administrator-defined profiles and require them to select one manually. Although this approach is simple and predictable, it places the responsibility for interpreting the catalog on the user. A profile name alone may not clearly indicate whether it is suitable for a particular workload, while selecting an environment based only on familiar software names does not necessarily guarantee that the environment satisfies explicit resource or capability requirements. Automating this decision is not straightforward either. A purely rule-based approach can provide deterministic and easily explainable decisions, but its ability to handle the variety of natural-language workload descriptions may be limited. Conversely, semantic retrieval or language-model-based approaches can improve the ability to interpret natural-language requests, but semantic relevance alone cannot guarantee that a candidate satisfies mandatory requirements or conforms to administrator-defined policies. The problem

addressed by this thesis is therefore the design of a context-aware, policy-bounded mechanism for recommending an appropriate JupyterHub resource profile and notebook image from an administrator-controlled catalog based on a user’s workload intent. The problem is deliberately narrower than general cloud resource optimization. The proposed system does not attempt to predict the exact runtime resource consumption of an arbitrary workload, replace Kubernetes scheduling, or dynamically optimize the entire cluster. Instead, it focuses on the decision that occurs before a notebook server is spawned: given a user’s description of the intended workload and a finite set of administrator-approved environments, how can the system identify and present candidates that are relevant while ensuring that explicit requirements and platform policies are respected?

## **1.2. Contributions and thesis overview**

The main contributions of this thesis are twofold. First, the thesis develops a context-aware resource profile selection approach that connects natural-language workload descriptions with a finite catalog of administrator-approved JupyterHub environments. The approach considers workload intent, explicit resource requirements, and the available environment catalog as relevant context during the selection process. This allows users to describe their computational needs at the workload level while keeping the actual resource environments within the set of configurations defined and controlled by the platform administrator. Second, the thesis proposes a multi-dimensional evaluation framework for assessing the resource profile selection approach. Rather than relying on a single recommendation metric, the evaluation considers several aspects of system behavior, including recommendation quality, robustness to variations in workload descriptions, resource-related behavior, notebook-image functionality, and practical integration with the JupyterHub environment. This broader evaluation is intended to provide a more comprehensive view of the effectiveness and practical applicability of the proposed approach.

The remainder of the thesis is organized into four chapters. Chapter 2 presents the technical and conceptual foundations required for the proposed work. It introduces JupyterHub and Kubernetes-based notebook environments, resource profiles and container images, workload intent, information retrieval, semantic retrieval, and recommendation-related concepts. These foundations establish the terminology and technical background used throughout the subsequent chapters. Chapter 3 presents the design of the proposed resource profile selection approach in detail. It describes the overall system architecture, workload-intent representation, candidate retrieval and ranking process, constraint handling, administrator-controlled environment catalog, and integration with the JupyterHub pre-spawn workflow. The chapter also discusses the main design decisions and defines the scope and boundaries of the proposed system. Chapter 4 describes the implementation and experimental methodology used to assess the proposed approach. It presents the experimental setup, evaluation scenarios, baselines, and evaluation metrics, followed by the experimental results and their discussion. The chapter also examines the limitations and scope of the evidence obtained from the experiments. Chapter 5 summarizes the work and its main findings, discusses the limitations of the current approach, and presents directions for future development and further evaluation. Overall, this thesis investigates the use of workload intent as an interface for selecting appropriate notebook environments in JupyterHub on Kubernetes. The central aim is to reduce the gap between user-level descriptions of computational tasks and infrastructure-level environment selection while maintaining administrator control and adherence to explicit platform constraints.

# Chapter 2.

## Background

This chapter introduces the technical concepts and methods required to understand the design of the proposed system. The discussion starts with interactive computational environments and their multi-user management through JupyterHub. It then describes Kubernetes-based execution, resource profiles, and notebook container images. The chapter subsequently introduces workload intent representation and the information retrieval techniques used for matching a natural-language request against a catalog of computational environments. Finally, constraint-based recommendation is discussed as the mechanism for ensuring that retrieved candidates satisfy explicit resource and environment requirements. The purpose of this chapter is to establish the concepts used later in the system design and evaluation. The implementation-specific architecture of the proposed system is presented separately in Chapter 3.

### 2.1. JupyterHub

Interactive computational environments are widely used for data analysis, scientific computing, machine learning, education, and software development. Instead of separating source code, execution, and results into independent tools, an interactive notebook allows these elements to be combined in a single computational document [12]. Jupyter Notebook is one of the most widely adopted environments following this model. At a basic level, a notebook server provides an interface through which a user can execute code interactively and inspect the resulting output. The Jupyter architecture separates the user-facing notebook interface from the kernel responsible for executing computations. Consequently, the computational environment can be configured independently of the browser-based interface. This separation is particularly useful in environments where users have different computational requirements. A lightweight educational notebook

may require only a basic Python environment, whereas a machine-learning workload may require additional libraries, larger amounts of memory, or specialized hardware accelerators. Therefore, providing an interactive notebook service at scale requires more than simply making the Jupyter interface available: the underlying execution environment must also be provisioned and isolated appropriately.

JupyterHub extends the Jupyter ecosystem from a single-user environment to a multi-user service [20]. It provides authentication, user management, and the creation of individual single-user notebook servers. The JupyterHub documentation describes the spawner as the component responsible for starting the single-user server and allocating its execution environment [9]. Conceptually, a JupyterHub deployment consists of several cooperating components, as illustrated in Figure 2.1. The Hub coordinates authentication through an Authenticator and manages user sessions recorded in a state database, while a Configurable HTTP Proxy routes incoming web requests either to the Hub for login operations or directly to the running notebook server of an authenticated user. A Spawner is responsible for creating and managing each user’s computational environment upon demand. The resulting architecture separates service management from the actual execution of notebook workloads.

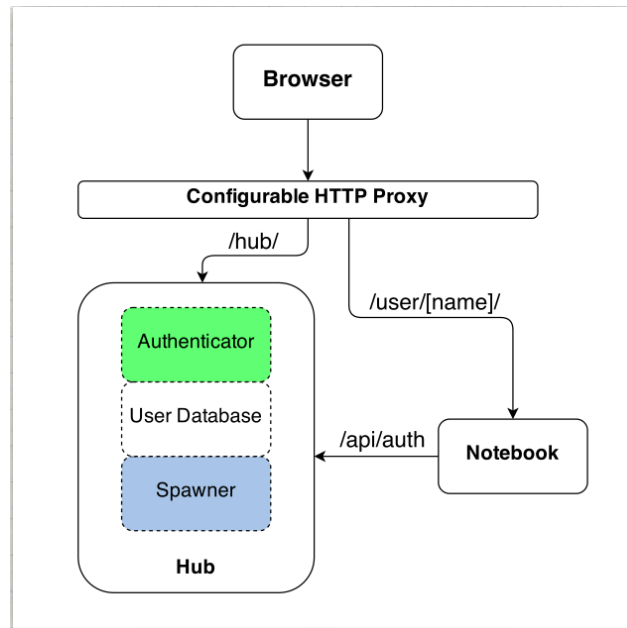


Figure 2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [9]

This distinction is important for resource-aware notebook provisioning. The Hub does not itself perform user computations. Instead, the spawner translates the desired user environment into the configuration required by the underlying execution platform, creating and tearing down instances as users arrive and leave.

JupyterHub also provides an abstract Spawner interface for starting, polling, and stopping single-user notebook servers [10]. Different spawner implementations can use different execution backends, including local operating system processes, container runtimes, and Kubernetes clusters. In a Kubernetes deployment, KubeSpawner creates each user’s notebook server as a dedicated Kubernetes workload [8, 24]. A spawner can also expose configuration options through the JupyterHub spawn form. Such options may represent CPU and memory allocations, container images, or a predefined combination of settings called a profile. JupyterHub’s documentation explicitly describes profiles as a mechanism through which users can select a bundle of configuration settings for their notebook server [10]. A profile therefore provides an abstraction between low-level infrastructure configuration and user choice. Rather than requiring users to understand the exact Kubernetes resource specification, an administrator can define a small set

of named environments such as *Small*, *Medium*, and *Large*. The user then selects the environment that appears appropriate for the workload. This mechanism provides an important foundation for automated environment recommendation. The recommendation problem can be formulated as selecting one of a finite set of administrator-defined environments rather than generating arbitrary infrastructure configurations dynamically.

## **2.2. Kubernetes-based notebook execution**

### **2.2.1. Containers and pods**

Kubernetes provides an orchestration layer for containerized applications. The basic execution unit relevant to a JupyterHub deployment is the Pod [15]. A Pod contains one or more containers that share storage volumes, a network namespace with a single IP address, and specifications on how to run the workloads. Figure 2.2 illustrates the conceptual structure of a Pod encapsulating containerized components and shared storage. For notebook platforms, a user’s single-user server is represented by a dedicated Pod where the container image determines the software environment available to the user, while the Kubernetes resource configuration determines the computational envelope allocated to the workload.

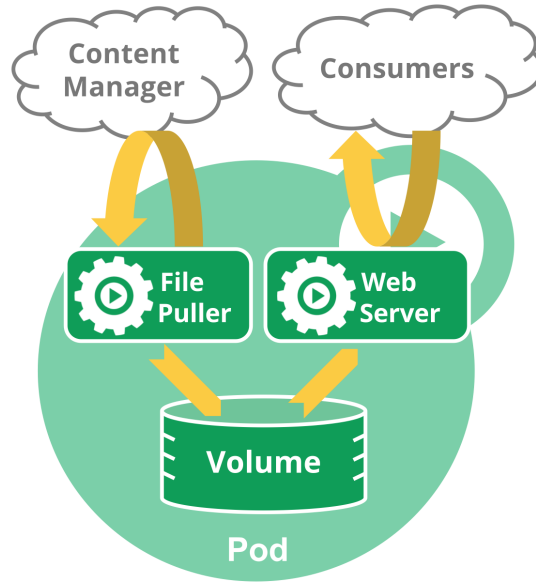


Figure 2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [15]

This separation establishes two complementary dimensions of environment selection. On the software dimension, the container image defines the runtime environment, pre-installed frameworks, domain-specific libraries, and underlying system dependencies. On the hardware resource dimension, the allocated CPU, memory, accelerator, and associated platform limits establish the computational envelope in which the workload executes. Consequently, an automated environment recommendation system must jointly consider both dimensions rather than treating resource sizing and software selection as unrelated choices.

### 2.2.2. CPU and memory resources

Kubernetes allows resource requests and limits to be specified for containers [15]. A resource request represents the amount of a resource guaranteed and considered by the Kubernetes scheduler during node placement, while a resource limit establishes an upper bound on resource consumption enforced at runtime [13]. For CPU and memory, these settings carry distinct operational implications. The Kubernetes scheduler uses resource requests when determining whether a Pod can be placed on a node without violating capacity constraints. In contrast, resource limits are enforced by kernel control groups:

CPU limits can lead to CPU throttling when exceeded, whereas memory limits cause out-of-memory (OOM) terminations when memory pressure forces a container beyond its configured boundary [15, 14]. Furthermore, aggregate namespace allocations can be restricted using ResourceQuota objects [16]. Consequently, resource configuration must balance two competing operational concerns. If a profile provides too few resources, a workload may fail through OOM kills or suffer severe performance degradation. If it provides substantially more resources than necessary, the environment consumes cluster capacity that could otherwise serve other concurrent users. For a multi-user notebook platform, resource-profile selection is thus a practical systems decision problem rather than merely a cosmetic interface preference.

### 2.2.3. Resource profiles

A resource profile is a predefined mapping from a human-readable environment choice to concrete infrastructure configurations. In the context of the proposed system, the repository defines three representative static profiles. The *Small* profile is intended for basic Python programming and lightweight notebook workloads, offering modest resource allocations to maximize cluster density. The *Medium* profile is configured for moderate data exploration, tabular data processing, and visualization tasks that require additional memory headroom. The *Large* profile is designed for compute- and memory-intensive workloads, such as local model training or iterative numerical experimentation. The profiles differ systematically in their CPU and memory allocations. For example, the repository configuration defines the Small profile with a CPU guarantee of 0.1 cores, a CPU limit of 0.5 cores, a memory guarantee of 256 MiB, and a memory limit of 384 MiB. The Medium and Large profiles provide progressively larger resource envelopes. These values represent administrative policy choices rather than universal definitions of workload scale. Their primary role is to provide a finite, administrator- controlled candidate space from which a recommendation can be selected. This distinction is central to the system developed in this thesis. The recommendation component does not invent arbitrary CPU or memory values. Instead, it selects an environment from the administrator-defined catalog and verifies the selected candidate against explicit policy and capacity constraints.

## 2.3. Notebook container images

A container image packages the software environment required to execute a workload, encompassing the base operating system runtime, language interpreters, scientific libraries, machine-learning frameworks, and supporting system packages [19]. As shown in Figure 2.3, container images follow a layered filesystem architecture where immutable, content-addressed read-only layers are stacked together, and a thin read-write layer is added at runtime when a container is instantiated [4]. This layered design enables efficient image distribution and local layer sharing across diverse notebook configurations.

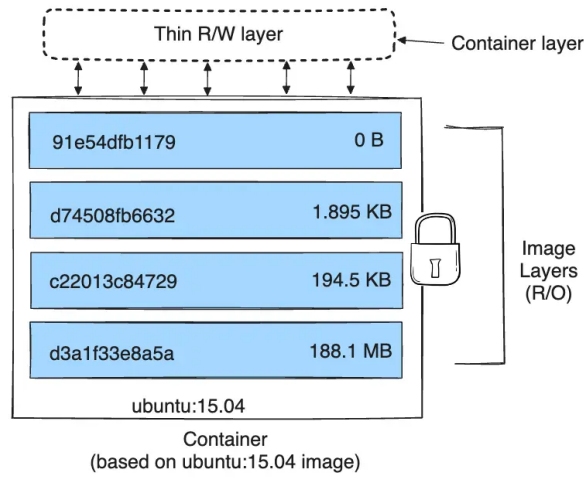


Figure 2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [4]

Different computational workloads inherently require different software stacks. A general programming task may require only a minimal Python runtime, while a data-analysis workflow requires packages such as NumPy, pandas, SciPy, and scikit-learn. Deep-learning workloads instead depend on specialized frameworks such as PyTorch or TensorFlow alongside accelerator drivers and CUDA runtime libraries. The proposed system models an execution environment as a coupled pair consisting of an approved resource profile and an administrator-reviewed notebook image, allowing the recommendation process to account simultaneously for computational resources and software dependencies.

Allowing an automated system to freely generate or pull arbitrary container image references introduces significant operational, security, and stability risks. An unvetted image may contain malicious code, outdated software, or incompatible dependencies that disrupt cluster operation. For this reason, the platform relies on an administrator-owned image catalog. The representative catalog entries include `minimal-python` for general Python programming and lightweight workloads, `scipy-data-science` for scientific computing, visualization, and conventional machine learning, `pytorch-deep-learning` for PyTorch-based deep-learning tasks, and `tensorflow-deep-learning` for TensorFlow- and Keras-based workflows. Each catalog entry is annotated with structured metadata detailing supported task types, frameworks, libraries, hardware capabilities, and suitability tags. This metadata provides a searchable semantic representation of the available environments, while also establishing an authoritative trust boundary: the recommendation algorithm is restricted to selecting among administrator-reviewed candidates rather than producing unverified container references.

## 2.4. Workload intent

Traditional resource-profile selection exposes infrastructure concepts directly to the end user. A user must determine whether a workload requires a small, medium, or large environment and separately identify which container image contains the appropriate software packages. This approach assumes that users can accurately translate their computational requirements into infrastructure specifications. In practice, users describe their goals in terms of the task they wish to perform. For example, a user may describe a workload as “I want to train a PyTorch model on a large dataset.” Such a description conveys information regarding the computational task, framework, and relative data scale, but it does not specify concrete Kubernetes CPU or memory allocations. The central abstraction used by the proposed system is therefore *workload intent*, drawing on the principles of intent-based computing where high-level user goals are translated into system configurations [1, 17, 23]. Workload intent provides a structured representation of what

the user wants to accomplish and what operational properties the resulting computational environment must satisfy.

Natural-language expressions provided by human users are inherently heterogeneous, unstructured, and ambiguous. A user describing a computational goal typically focuses on domain objectives (such as the machine learning framework, analytical procedure, or estimated dataset size) rather than low-level infrastructure attributes. Intent-based computing formalizes this paradigm by decoupling user intent specification from underlying execution configuration [1, 17]. Under this framework, high-level intent is translated into a validated, declarative contract that captures functional goals, explicit constraints, and user preferences. By establishing a structured intermediate representation, intent-based systems isolate the stochastic nature of natural language interpretation from deterministic execution engines. This structured bridge enables automated systems to parse user goals, validate semantic consistency, and apply rigorous policy enforcement without giving natural-language interfaces direct access to infrastructure provisioning mechanisms [23].

## 2.5. Information retrieval

The environment catalog used by the proposed system is a finite collection of administrator-defined computational environments. Each environment is associated with structured metadata describing properties such as supported workload types, frameworks, libraries, capabilities, and resource characteristics. The recommendation problem can therefore be viewed as a search problem over this catalog: given a user’s workload description, the system must identify the environments whose metadata is most relevant to the request. This perspective motivates the use of *information retrieval* (IR). Rather than requiring every possible natural-language expression to be explicitly encoded as a rule, retrieval allows the system to search the existing environment catalog and produce a ranked set of potentially relevant candidates.

Let

$$\mathcal{D} = \{d_1, d_2, \dots, d_n\}$$

represent the administrator-defined environment catalog and let  $q$  represent the user’s workload intent. A retrieval function produces a ranked list

$$R(q) = (d_{(1)}, d_{(2)}, \dots, d_{(k)}),$$

where candidates appearing earlier in the list are considered more relevant to the query.

The use of retrieval is particularly appropriate because the system does not need to construct a new environment for every request. Instead, it searches within a controlled catalog and identifies candidates that already exist. This also preserves the administrator-defined boundary of the system: retrieval determines which existing environments are relevant, while subsequent constraint evaluation determines which of them are actually feasible. A straightforward approach would be to match the words in the user’s request directly against the metadata of available environments. Although simple and interpretable, exact keyword matching is sensitive to differences in vocabulary. For example, a user may describe a workload as “training a neural network”, while an environment may be described using terms such as “deep learning”, “PyTorch”, or “model training”. These expressions may refer to related workloads without sharing exactly the same words. The opposite problem can also occur. Technical terms such as “PyTorch”, “TensorFlow”, or “CUDA” carry specific meanings and should not necessarily be treated as interchangeable with loosely related concepts. Consequently, an effective retrieval mechanism should provide both lexical precision for explicit technical terminology and tolerance to variations in natural-language expression. This motivates the use of two complementary retrieval approaches: sparse lexical retrieval and dense semantic retrieval.

### 2.5.1. Sparse retrieval and BM25

Sparse information retrieval methods represent queries and candidate documents primarily through their lexical features. They are particularly effective when important concepts are explicitly expressed using the same terms in the query and candidate metadata.

BM25 is a widely used probabilistic ranking function [22]. Given a query  $Q$  and document  $D$ , its score can be expressed as

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1 \left(1 - b + b \frac{|D|}{\text{avgdl}}\right)},$$

where  $f(t, D)$  is the frequency of term  $t$  in document  $D$ ,  $|D|$  is the document length,  $\text{avgdl}$  is the average document length across the corpus,  $\text{IDF}(t)$  is the inverse document frequency of term  $t$ , and  $k_1$  and  $b$  control term-frequency saturation and length normalization, respectively. BM25 is useful for the proposed environment catalog because explicit technical vocabulary can be highly informative. For example, a request containing “pandas” can be matched directly against an environment whose metadata contains “pandas”. Similarly, an explicit mention of “PyTorch” can provide strong evidence for a PyTorch-oriented environment. However, BM25 primarily relies on lexical overlap. If a user and an environment describe the same concept using substantially different vocabulary, their lexical similarity may be low. Therefore, BM25 alone does not fully address the variability of natural-language workload descriptions.

### 2.5.2. Dense retrieval

Dense retrieval addresses vocabulary mismatch by representing queries and candidate environments as vectors in a continuous embedding space [21, 11, 18]. Let  $\mathbf{q} = f(q)$  be the embedding of a query and  $\mathbf{d} = f(d)$  be the embedding of a candidate environment. A common similarity measure is cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}.$$

Candidates with larger similarity values are considered semantically closer to the query.

The main advantage of dense retrieval is its ability to capture semantic relationships beyond exact word overlap. For example, a query describing “neural-network training” may be considered semantically related to an environment described as supporting “deep learning” and “model training”, even if the two descriptions do not share all of their terms. Dense retrieval alone, however, also has limitations for this application. Embedding similarity may treat technically related expressions as similar even when an exact distinction is important. For example, different machine learning frameworks may be semantically related but are not necessarily interchangeable when a user explicitly requires one particular framework. The ranking produced by dense retrieval can also depend strongly on the embedding model and the representation of the candidate metadata. Therefore, dense retrieval is useful for discovering semantic relationships, but it should not be treated as the sole mechanism for interpreting explicit technical requirements.

### **2.5.3. Hybrid retrieval with reciprocal rank fusion**

The characteristics of the environment recommendation problem suggest that neither sparse nor dense retrieval is sufficient on its own. Sparse retrieval provides strong lexical sensitivity. This is valuable when a user explicitly mentions technical entities such as a programming framework, library, or capability. Dense retrieval provides semantic flexibility and can help identify relevant environments when the user’s wording differs from the wording used in the catalog. These properties are complementary. Sparse retrieval is effective for explicit terminology and exact technical concepts, dense retrieval is effective for semantic similarity and variation in natural-language descriptions, and hybrid retrieval combines these signals to obtain a broader candidate set while retaining sensitivity to explicit technical vocabulary. Hybrid retrieval is therefore selected as the retrieval strategy for the proposed system. The goal is not to allow semantic similarity to override explicit requirements, but to improve the identification of relevant candidates before deterministic constraint evaluation. This distinction is important. Retrieval is responsible for answering “*Which environments appear relevant to this workload?*”, whereas the subsequent constraint layer answers “*Which of these environments are actually allowed and feasible?*”. Separating these two questions prevents a high semantic similarity score

from being treated as sufficient evidence that an environment satisfies a mandatory requirement.

Let  $r_s(d)$  and  $r_d(d)$  denote the ranks assigned to candidate  $d$  by the sparse and dense retrieval methods, respectively. Their rankings can be combined using Reciprocal Rank Fusion (RRF) [2]:

$$\text{RRF}(d) = \sum_{m \in \{s,d\}} \frac{w_m}{k_{\text{RRF}} + r_m(d)},$$

where  $w_m$  is the weight assigned to retrieval method  $m$ , and  $k_{\text{RRF}}$  is a constant controlling the contribution of lower-ranked candidates.

RRF is suitable for combining the two retrieval channels because it operates on rankings rather than requiring their raw scores to be directly comparable. This is useful in this setting because BM25 scores and embedding similarities have different numerical scales and interpretations. The resulting hybrid ranking provides the candidate set for the next stage of the recommendation pipeline. Importantly, this ranking is not itself the final provisioning decision. Retrieved candidates are subsequently evaluated against hard resource and software constraints and may then be ordered using soft preferences.

## 2.6. Constraint-based recommendation

Information retrieval assesses lexical and semantic relevance, but relevance alone is insufficient for decision-support and resource provisioning systems. A candidate solution may demonstrate high textual affinity to a user query while failing to meet mandatory operational requirements, physical prerequisites, or administrative policies. For domains governed by explicit rules, recommendation systems must incorporate constraint satisfaction mechanisms to guarantee solution validity. Constraint-based recommendation represents a major paradigm within knowledge-based recommender systems, designed for complex problem domains where items must satisfy strict technical specifications, compatibility rules, and domain constraints [5]. Unlike collaborative filtering or pure content-based retrieval models that rely primarily on statistical correlation or historical

usage patterns, constraint-based approaches model the recommendation task by combining constraint satisfaction with multi-attribute utility optimization.

In constraint-based recommendation theory, domain criteria are partitioned into two fundamental categories. Hard constraints are strict, non-negotiable operational requirements that define the boundary of valid solutions. A candidate item must satisfy all hard constraints simultaneously to be considered feasible. In technical systems, hard constraints prevent severe runtime failures (such as resource exhaustion or missing runtime dependencies), because violations cannot be compensated for or traded off against high relevance scores. On the other hand, soft constraints and preferences are desirable but optional criteria that express user affinities, quality indicators, or secondary optimization goals. Soft preferences are evaluated across the set of verified feasible solutions using multi-attribute utility functions, guiding the ranking of viable candidates without compromising feasibility [6, 7]. Structuring the decision-support pipeline into a two-phase architecture candidate retrieval followed by deterministic constraint evaluation and utility ranking ensures both search efficiency and operational correctness. Candidate retrieval efficiently narrows a large catalog down to a promising subset, while subsequent constraint filtering guarantees that only viable, policy-compliant candidates proceed to final ranking and deployment.

# Chapter 3.

## Proposed Method

### 3.1. Overview

This chapter presents the design and formulation of the proposed method for intent- and context-aware notebook environment selection in JupyterHub on Kubernetes. The method addresses the challenge of translating an end user’s natural-language workload description into an appropriate, policy-compliant computational environment without requiring manual exploration of container images, hardware resource envelopes, or platform configurations. The proposed system, designated as P2, is structured as a multi-stage recommendation and enforcement pipeline. Rather than delegating deployment decisions to an unconstrained generative model, the architecture strictly decouples semantic interpretation from infrastructure authority. Natural-language text is used exclusively to extract structured semantic facts, whereas candidate retrieval, feasibility filtering, ranking, and runtime binding operate strictly over a bounded, administrator-owned candidate catalog.

The overall operational transformation executed by the P2 pipeline can be expressed as a sequence of deterministic and inspectable transformations:

$$x \longrightarrow I \longrightarrow (L_s, L_d) \longrightarrow L_f \longrightarrow F \longrightarrow c^* \longrightarrow e^*,$$

where  $x$  denotes the untrusted natural-language workload request;  $I$  represents the validated, normalized structured intent;  $L_s$  and  $L_d$  are the candidate shortlists generated by sparse lexical retrieval and dense semantic retrieval, respectively;  $L_f$  is the fused candidate ranking produced by Reciprocal Rank Fusion;  $F$  represents the feasible candidate subset remaining after deterministic hard constraint evaluation;  $c^*$  is the top-ranked candidate

identifier selected by deterministic preference ranking; and  $e^*$  is the authoritative spawn configuration resolved from the administrator-controlled catalog and enforced by the JupyterHub pre-spawn hook.

Figure 3.1 illustrates the end-to-end architecture of the proposed system, highlighting the modular decomposition and the clear separation of trust boundaries across functional components.

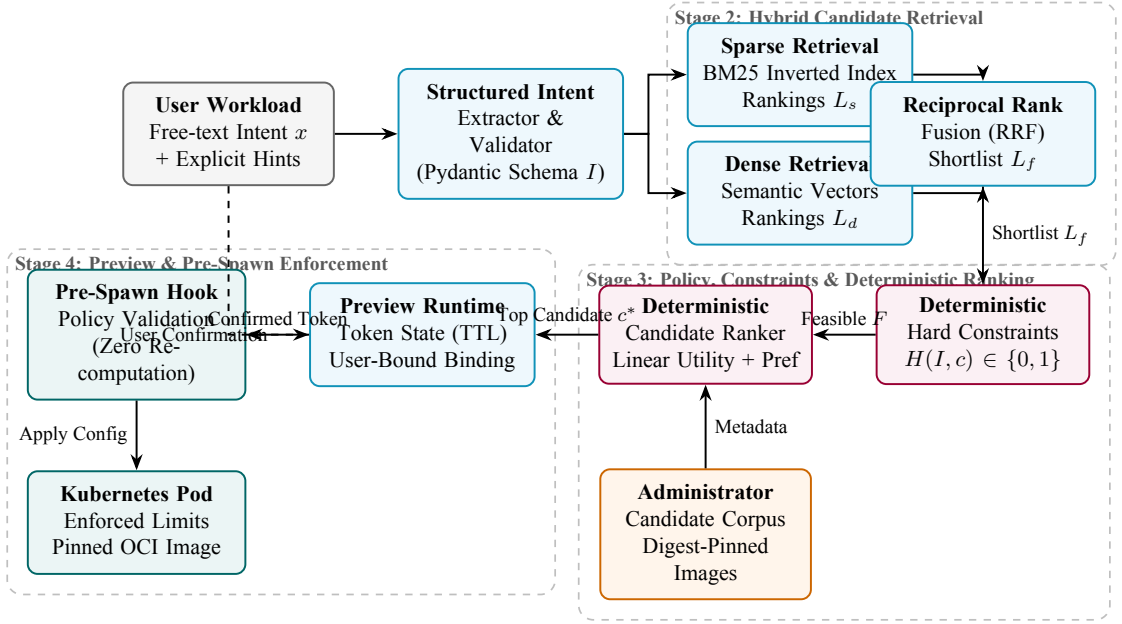


Figure 3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture

The P2 architecture organizes these operations across five distinct logical components structured along a feed-forward workflow. First, the Intent Interface and Extractor captures free-form workload descriptions and explicit user hints, parsing them into a strongly typed, schema-validated StructuredIntent contract ( $I$ ). Next, the Candidate Catalog and Hybrid Retriever queries verified pairings of resource profiles and container images via parallel sparse lexical and dense semantic channels, generating candidate shortlists  $L_s$  and  $L_d$  that are subsequently fused into  $L_f$  using Reciprocal Rank Fusion. The Constraint Evaluator and Deterministic Ranker then deterministically prunes candidates failing mandatory resource, accelerator, or capability constraints to establish the feasible candidate set  $F$ , subsequently ordering the remaining candidates through linear rank utility,

soft user preferences, and resource cost efficiency to determine the top-ranked candidate  $c^*$ . Following candidate selection, the Policy Validator and Preview Runtime validates  $c^*$  against authoritative administrator-governed allowlists and persists the decision as a time-limited preview state bound to an ephemeral, single-use cryptographic token. Finally, upon explicit user confirmation, the JupyterHub Pre-Spawn Integration Hook intercepts the notebook launch request, validates token provenance, and enforces the static container image and resource limits in KubeSpawner without performing runtime model inference. This component architecture ensures that recommendations remain advisory until explicitly approved by the user, and that the configuration applied to the Kubernetes Pod strictly matches the validated preview, preventing Time-of-Check to Time-of-Use (TOCTOU) discrepancies.

## 3.2. Structured intent representation

Natural-language workload descriptions exhibit high semantic variability, combining general computational descriptions, software preferences, and strict hardware constraints. Treating these heterogeneous elements uniformly within an unstructured query string creates an undesirable coupling between uncertain language interpretation and strict operational enforcement. To resolve this challenge, P2 introduces an intermediate structured data contract designated as StructuredIntent. The StructuredIntent contract acts as an immutable boundary between natural language understanding and candidate ranking. It structures extracted workload properties into functional dimensions: task categories, required features, preferred features, forbidden features, required frameworks, preferred frameworks, required libraries, preferred libraries, explicit resource constraints, ambiguity diagnostics, normalized query text, and extraction provenance metadata. The distinction between required and preferred attributes is central to the system's correctness. A required feature or framework is treated as a strict feasibility condition that must be demonstrated by candidate metadata; failure to satisfy a required attribute results in immediate candidate pruning. In contrast, a preferred feature represents a non-mandatory user affinity that contributes positively to the soft ranking score of candidates that have already satisfied

all mandatory requirements. Table 3.1 outlines the schema specification and validation rules governing the StructuredIntent data model.

**Table 3.1. Specification of the StructuredIntent data contract and normalization rules**

| Field Name            | Data Type       | Normalization Rule            | Evaluation Role                             |
|-----------------------|-----------------|-------------------------------|---------------------------------------------|
| task_types            | Tuple[TaskType] | Enum validation, deduplicated | Sparse/dense query context                  |
| required_features     | Tuple[str]      | NFKC, case-folded, sorted     | Hard constraint filter ( $H$ )              |
| preferred_features    | Tuple[str]      | NFKC, case-folded, sorted     | Soft preference score ( $S_{\text{pref}}$ ) |
| forbidden_features    | Tuple[str]      | NFKC, case-folded, sorted     | Hard exclusion predicate ( $H$ )            |
| required_frameworks   | Tuple[str]      | NFKC, case-folded, sorted     | Hard constraint filter ( $H$ )              |
| preferred_frameworks  | Tuple[str]      | NFKC, case-folded, sorted     | Soft preference score ( $S_{\text{pref}}$ ) |
| required_libraries    | Tuple[str]      | NFKC, case-folded, sorted     | Hard constraint filter ( $H$ )              |
| preferred_libraries   | Tuple[str]      | NFKC, case-folded, sorted     | Soft preference score ( $S_{\text{pref}}$ ) |
| gpu_requirement       | GPURequirement  | Enum (required/preferred/not) | Hard constraint / preference                |
| minimum_cpu_cores     | Optional[float] | Finite non-negative float     | Hard lower-bound constraint                 |
| minimum_memory_gb     | Optional[float] | Finite non-negative float     | Hard lower-bound constraint                 |
| dataset_size_gb       | Optional[float] | UI precedence, finite float   | Hard memory headroom bound                  |
| ambiguities           | Tuple[str]      | Printable characters, sorted  | Diagnostic explanation                      |
| normalized_query      | str             | Whitespace collapsed          | Sparse BM25 retrieval token                 |
| extraction_provenance | Provenance      | Extractor ID, mode, hash      | Security and audit tracking                 |

Algorithm 3.1 formalizes the structured intent extraction, normalization, and validation procedure. It guarantees that raw natural-language inputs and explicit UI parameters are deterministically canonicalized into an immutable, schema-compliant contract prior to candidate retrieval.

---

**Algorithm 3.1. Structured intent extraction, normalization, and validation procedure**

---

**Require:** Raw user request string  $x$ , explicit UI parameters  $H_{\text{user}} =$

$\langle \text{min\_cpu}, \text{min\_ram}, \text{dataset\_size}, \text{gpu} \rangle$

**Ensure:** Validated, immutable structured intent contract  $I$

```
1:  $I_{\text{raw}} \leftarrow \text{ExtractSemantics}(x)$   $\triangleright$  Structured LLM parser or regex fallback
2:  $I_{\text{norm}} \leftarrow \text{CanonicalizeUnicodeAndText}(I_{\text{raw}})$ 
3:  $I_{\text{norm}}.\text{req\_feats} \leftarrow \text{DeduplicateAndSort}(I_{\text{norm}}.\text{req\_feats})$ 
4:  $I_{\text{norm}}.\text{pref\_feats} \leftarrow \text{DeduplicateAndSort}(I_{\text{norm}}.\text{pref\_feats}) \setminus I_{\text{norm}}.\text{req\_feats} \triangleright$  Required takes precedence
5: if  $I_{\text{norm}}.\text{req\_feats} \cap I_{\text{norm}}.\text{forbid\_feats} \neq \emptyset$  then
6:   return error(InvalidContract: Contradictory required and forbidden features)
7: end if
8:  $R \leftarrow \text{OverrideResourceConstraints}(I_{\text{norm}}.\text{resource\_constraints}, H_{\text{user}})$   $\triangleright$  UI hints override extracted values
9:  $I \leftarrow \text{FreezeContract}(I_{\text{norm}}, R, \text{schema\_version} = "1.0.0")$ 
10: return  $I$ 
```

---

Prior to candidate retrieval, all textual and categorical fields within the structured intent undergo systematic normalization. The system executes Unicode Normalization Form KC (NFKC) conversion, collapses consecutive whitespace, and applies case-folding. Semantic collections, including framework and library names, are deduplicated and lexicographically sorted. This canonicalization ensures that superficial formatting variations in user prompts do not alter recommendation behavior (e.g., canonicalizing "Scikit-Learn", "scikit\_learn", and " scikit-learn " to identical internal representations). Furthermore, contradictory specifications are resolved deterministically: if a capability is declared as both required and preferred, the required classification takes precedence, whereas overlapping required and forbidden attributes trigger an immediate contract validation error. Explicit numerical values supplied directly through the user interface are granted strict precedence over quantities inferred from free-form text. If

a user specifies a dataset size of 4.0 GB in a numeric input field, this value is adopted authoritatively as `I.dataset_size_gb`, overriding ambiguous textual references.

The structured intent extractor is strictly constrained to emit semantic descriptions of the workload rather than infrastructure configuration directives. Specifically, the schema forbids the inclusion of Kubernetes resource objects, raw container image registries, or arbitrary profile names. This architectural boundary reduces the attack surface and prevents natural language prompts from injecting unauthorized deployment configurations or arbitrary container specifications. The system incorporates dual extraction mechanisms: an external structured extraction interface (e.g., supported by large language models) and a deterministic, regex-based local fallback extractor. When an external extraction service experiences network failure, timeout, or emits malformed output, the system automatically degrades to deterministic local extraction. The provenance of the extraction mode is recorded in the `extraction_provenance` metadata field and preserved throughout the recommendation trace for auditing.

### **3.3. Hybrid candidate retrieval**

#### **3.3.1. Dual-channel candidate retrieval pipeline**

To operationalize candidate discovery over the administrator catalog  $C$ , the P2 recommendation engine deploys a dual-channel hybrid retriever combining complementary sparse lexical and dense semantic pathways. Figure 3.2 depicts the data flow and component interactions of this retrieval and rank fusion stage.

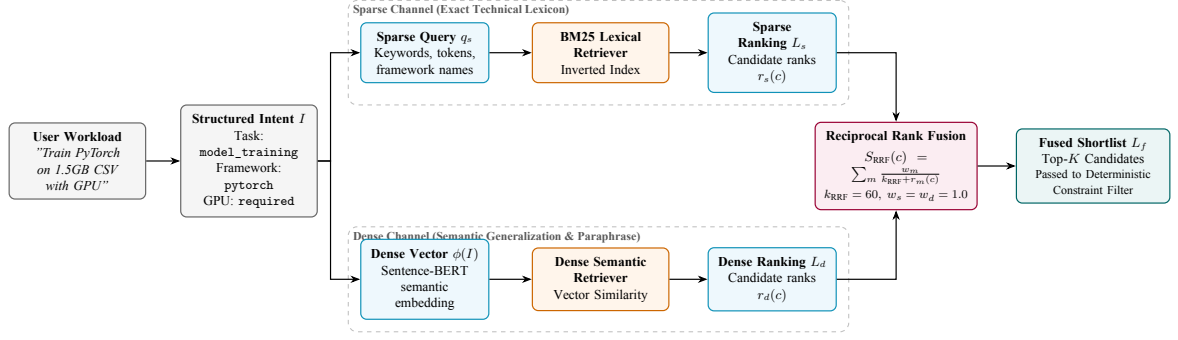


Figure 3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion

The two retrieval mechanisms execute in parallel:

$$L_s = \text{BM25}(q_s, C), \quad L_d = \text{Dense}(\phi(I), C),$$

where  $q_s$  is the synthesized sparse lexical query constructed from the structured intent,  $\phi(I)$  is the dense vector embedding of the intent, and  $L_s$  and  $L_d$  are the resulting ranked candidate shortlists.

### 3.3.2. Sparse lexical query construction and execution

The sparse retrieval module applies BM25 to index and search candidate environment documents, configured with standard hyperparameters  $k_1 = 1.5$  and  $b = 0.75$ .

The key mechanism of this stage is the deterministic transformation of the validated StructuredIntent  $I$  into an expanded lexical query  $q_s$ . Rather than passing only the raw natural-language prompt  $x$ , the query builder synthesizes a structured lexical string by aggregating multiple semantic fields:

$$q_s = \text{Concat} \left( x, I.\text{normalized\_query}, \text{Tokens}(I.\text{task\_types}), \right. \\ \left. \text{Tokens}(I.\text{req\_frameworks} \cup I.\text{pref\_frameworks}), \right. \\ \left. \text{Tokens}(I.\text{req\_libs} \cup I.\text{pref\_libs}) \right).$$

Query  $q_s$  undergoes NFKC normalization, lowercasing, punctuation stripping, and token deduplication. Executing  $q_s$  against the candidate corpus yields an ordered list  $L_s$  ranked by BM25 score, ensuring that exact technical keywords (such as specific library names, framework identifiers, and API terms) are matched with high fidelity.

### 3.3.3. Dense semantic indexing and query inference

The dense retrieval channel captures conceptual semantic affinities when user phrasing differs from catalog keywords. Candidate documents  $c \in C$  are mapped offline into a continuous embedding space using a pre-trained transformer model (all-MiniLM-L6-v2), producing normalized vectors  $\phi(c) \in \mathbb{R}^D$  serialized into an immutable index artifact.

At recommendation time, the normalized structured intent  $I$  is encoded into query embedding  $\phi(I) \in \mathbb{R}^D$ . Dense retrieval computes the cosine similarity between  $\phi(I)$  and candidate vectors  $\phi(c)$ , yielding an ordered candidate list  $L_d$  sorted by descending semantic similarity.

### 3.3.4. Reciprocal rank fusion implementation

To synthesize rankings from sparse and dense channels without relying on uncalibrated raw score interpolation, P2 implements Reciprocal Rank Fusion (RRF). For each candidate  $c \in \mathcal{C} = \text{Candidates}(L_s) \cup \text{Candidates}(L_d)$ , the fused score is defined as:

$$\text{RRF}(c) = \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\text{RRF}} + \text{rank}(c, L_m)},$$

where  $k_{\text{RRF}} = 60.0$ , and channel weights are set to  $w_s = 1.0$  and  $w_d = 1.0$ .

Algorithm 3.2 details the fusion and shortlist truncation procedure. The fused ranking is truncated to a top- $K$  candidate shortlist  $L_f$  ( $K = 10$ ), which is subsequently passed to deterministic constraint evaluation. Score ties among candidates are broken deterministically by ascending alphanumeric candidate identifier.

---

**Algorithm 3.2. Reciprocal Rank Fusion for hybrid candidate retrieval**

---

**Require:** Ranked sparse list  $L_s$ , ranked dense list  $L_d$ , fusion constant  $k_{\text{RRF}}$ , weights  $w_s, w_d$ , shortlist capacity  $K$

**Ensure:** Fused candidate shortlist  $L_f = (c_{(1)}, c_{(2)}, \dots, c_{(K)})$

- 1:  $\mathcal{C} \leftarrow \text{Candidates}(L_s) \cup \text{Candidates}(L_d)$
  - 2: **for** each candidate  $c \in \mathcal{C}$  **do**
  - 3:      $\text{RRF}(c) \leftarrow \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\text{RRF}} + \text{rank}(c, L_m)}$
  - 4: **end for**
  - 5:  $L_f \leftarrow \text{TopK} \left( \mathcal{C}, K, \text{key} = c \mapsto (-\text{RRF}(c), c) \right) \triangleright \text{Descending score, alphanumeric tie-breaker}$
  - 6: **return**  $L_f$
- 

### 3.4. Deterministic constraint evaluation

Following candidate retrieval, every candidate in shortlist  $L_f$  is evaluated against mandatory technical constraints. A hard constraint represents an uncompromising operational condition: if a candidate fails even a single constraint, it is immediately pruned regardless of its retrieval score.

Formally, let  $h_j(I, c) \in \{0, 1\}$  represent the binary compliance outcome of the  $j$ -th hard constraint predicate for candidate  $c$  under structured intent  $I$ . The overall feasibility predicate  $H(I, c)$  is defined as their logical conjunction:

$$H(I, c) = \prod_{j=1}^J h_j(I, c).$$

A candidate  $c$  is deemed feasible if and only if  $H(I, c) = 1$ . The feasible candidate set  $F$  is defined as:

$$F = \left\{ c \in L_f \mid H(I, c) = 1 \right\}.$$

Table 3.2 summarizes the hard constraint rules, mathematical predicates, operational enforcement conditions, and diagnostic error codes. Figure 3.3 illustrates the sequential pruning decision tree executed for each candidate.

**Table 3.2. Deterministic hard constraint evaluation rules and pruning conditions**

| Constraint         | Predicate $h_j(I, c)$                                                                                            | Enforcement Condition                                        | Pruning Diagnostic   |
|--------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------|
| GPU Requirement    | $\mathbb{I}(I.\text{gpu} = \text{req} \implies c.\text{has\_gpu})$                                               | Requires accelerator device presence in catalog profile      | gpu_unavailable      |
| CPU Cores          | $\mathbb{I}(c.\text{cpu\_limit} \geq I.\text{min\_cpu})$                                                         | Profile CPU limit must meet or exceed intent minimum         | insufficient_cpu     |
| Memory Capacity    | $\mathbb{I}(c.\text{mem\_limit} \geq I.\text{min\_ram})$                                                         | Profile memory limit must meet or exceed intent minimum      | insufficient_memory  |
| Dataset Headroom   | $\mathbb{I}(c.\text{mem\_limit} > I.\text{dataset\_size})$                                                       | Profile RAM must exceed estimated dataset footprint          | dataset_overflow     |
| Required Features  | $\mathbb{I}(I.\text{req\_feats} \subseteq c.\text{capabilities})$                                                | Candidate must advertise all requested mandatory tools       | missing_capability   |
| Forbidden Features | $\mathbb{I}(I.\text{forbid\_feats} \cap c.\text{capabilities} = \emptyset)$                                      | Candidate must not contain any explicitly prohibited feature | forbidden_capability |
| Catalog Allowlist  | $\mathbb{I}(c.\text{profile} \in \mathcal{P}_{\text{allow}} \wedge c.\text{img} \in \mathcal{I}_{\text{allow}})$ | Candidate must belong to active administrator policy         | unapproved_candidate |

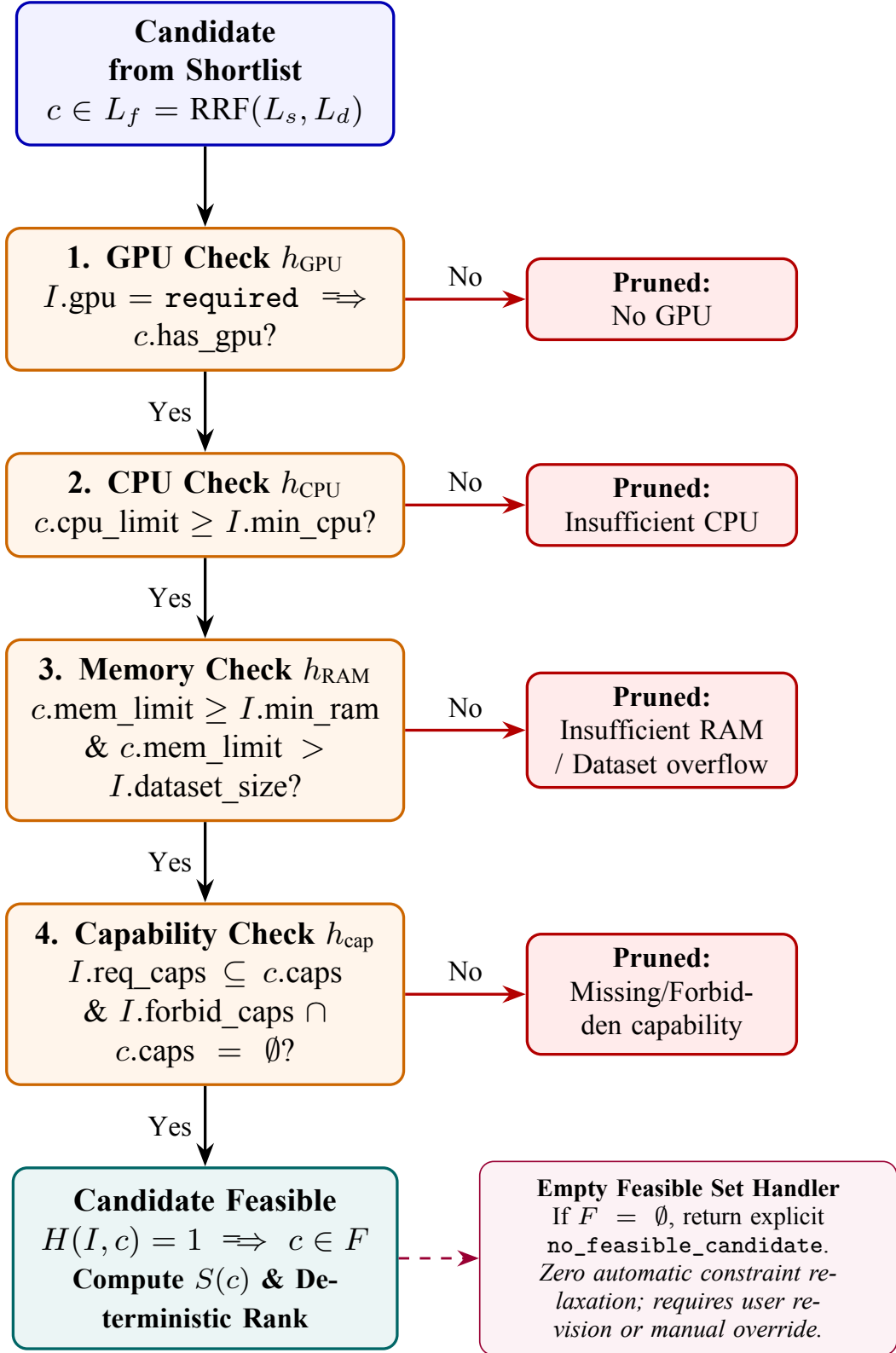


Figure 3.3. Deterministic hard constraint evaluation decision tree and pruning logic

Hardware accelerator, CPU, and memory requirements are enforced through strict binary boundary predicates:

$$h_{\text{GPU}}(I, c) = \mathbb{I}(I.\text{gpu\_requirement} \neq \text{required} \vee c.\text{has\_gpu} = \text{true}),$$

$$h_{\text{CPU}}(I, c) = \mathbb{I}(c.\text{cpu\_limit} \geq I.\text{min\_cpu}),$$

$$h_{\text{RAM}}(I, c) = \mathbb{I}(c.\text{mem\_limit} \geq I.\text{min\_ram} \wedge c.\text{mem\_limit} > I.\text{dataset\_size\_gb}).$$

If an intent specifies a mandatory GPU, any CPU-only candidate is strictly pruned. Similarly, profile memory limits must satisfy both the requested minimum memory and exceed the declared dataset size, preventing immediate runtime Out-Of-Memory (OOM) pod terminations.

Software capabilities and architectural constraints undergo set-theoretic verification against candidate catalog declarations:

$$h_{\text{caps}}(I, c) = \mathbb{I}(I.\text{req\_feats} \subseteq c.\text{capabilities}) \cdot \mathbb{I}(I.\text{forbid\_feats} \cap c.\text{capabilities} = \emptyset).$$

P2 adopts a conservative metadata policy: if a candidate does not explicitly declare a required capability in its verified catalog record, the capability is treated as absent. Semantic proximity in embedding space is never accepted as a substitute for unadvertised software packages or drivers.

A central design property of P2 is treating an empty feasible set  $F = \emptyset$  as a valid, first-class system outcome. When constraint evaluation determines that no retrieved candidate satisfies all mandatory requirements, the system does not relax constraints or guess an alternative environment. Instead, P2 halts execution and returns an explicit `no_feasible_candidate` result. The user interface displays the specific constraints that caused infeasibility (e.g., missing GPU availability or dataset size exceeding profile limits). The user is prompted to refine their workload description or manually select an administrator-allowlisted profile. This defensive design converts silent under-provisioning errors into visible, actionable feedback.

## 3.5. Deterministic candidate ranking

### 3.5.1. Multi-Criteria deterministic scoring formulation

All feasible candidates  $c \in F$  are evaluated by a multi-criteria scoring function balancing retrieval relevance, soft user preferences, and resource cost efficiency. The ranking score  $S(c) \in [0, 1]$  is formulated as:

$$S(c) = w_{\text{ret}} \cdot U(r_f(c)) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c)),$$

where  $U(r_f(c))$  denotes the linear rank utility derived from fused retrieval rank  $r_f(c)$ ,  $S_{\text{pref}}(I, c) \in [0, 1]$  measures soft preference alignment, and  $\text{Cost}(c) \in [0, 1]$  represents the candidate's normalized resource footprint. The weights are configured as:

$$w_{\text{ret}} = 0.60, \quad w_{\text{pref}} = 0.25, \quad w_{\text{res}} = 0.15,$$

satisfying  $w_{\text{ret}} + w_{\text{pref}} + w_{\text{res}} = 1.00$ .

### 3.5.2. Linear rank utility and resource cost modeling

The rank utility function  $U(r)$  converts ordinal retrieval positions into a linear prior:

$$U(r) = \max(0.0, 1.0 - \delta_{\text{rank}} \cdot (r - 1)),$$

where the rank decay parameter  $\delta_{\text{rank}} = 0.10$ . Each retrieval rank drop incurs a constant score penalty of  $w_{\text{ret}} \cdot \delta_{\text{rank}} = 0.60 \times 0.10 = 0.06$ . This linear formulation enables exact quantification of promotion distance: soft preferences alone ( $w_{\text{pref}} = 0.25$ ) can promote a candidate by up to four ranks ( $0.25/0.06 \approx 4.17$ ), while combined preference and resource efficiency ( $0.25 + 0.15 = 0.40$ ) can promote a highly suitable candidate from rank 7 to rank 1.

The resource cost metric  $\text{Cost}(c)$  formalizes the systems principle of minimal sufficiency: among multiple feasible candidates, the platform prefers the candidate with

the smallest resource envelope, conserving cluster capacity for concurrent tenants [15, 3]:

$$\text{Cost}(c) = \frac{\alpha_{\text{cpu}} \cdot \frac{c.\text{cpu}}{\text{max CPU}} + \alpha_{\text{ram}} \cdot \frac{c.\text{ram}}{\text{max RAM}} + \alpha_{\text{gpu}} \cdot \frac{c.\text{gpu}}{\text{max GPU}}}{\alpha_{\text{cpu}} + \alpha_{\text{ram}} + \alpha_{\text{gpu}}},$$

where weights are set to  $\alpha_{\text{cpu}} = 1.0$ ,  $\alpha_{\text{ram}} = 1.0$ , and  $\alpha_{\text{gpu}} = 2.0$ , reflecting accelerator scarcity.

Algorithm 3.3 formalizes candidate evaluation, multi-criteria scoring, and tie-breaking. Rather than re-evaluating individual constraint conditions, the algorithm delegates feasibility verification to the constraint evaluator  $\text{IsFeasible}(I, c)$ . Deterministic tie-breaking is enforced by sorting candidates using a four-element total order: descending score ( $-S(c)$ ), ascending resource cost ( $\text{Cost}(c)$ ), ascending retrieval rank ( $r$ ), and ascending lexicographical candidate identifier.

---

**Algorithm 3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking**

---

**Require:** Fused candidate shortlist  $L_f$ , structured intent  $I$ , weights  $(w_{\text{ret}}, w_{\text{pref}}, w_{\text{res}})$ , rank decay  $\delta_{\text{rank}}$

**Ensure:** Recommended candidate identifier  $c^*$  (or  $\emptyset$  if no candidate is feasible)

```
1:  $\mathcal{F} \leftarrow \emptyset$ 
2: for each candidate  $c \in L_f$  do
3:   if  $\neg \text{IsFeasible}(I, c)$  then
4:     continue ▷ Prune candidate violating hard constraints
5:   end if
6:    $r \leftarrow \text{rank}(c, L_f)$ 
7:    $U(r) \leftarrow \max(0.0, 1.0 - \delta_{\text{rank}} \cdot (r - 1))$ 
8:    $S(c) \leftarrow w_{\text{ret}} \cdot U(r) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c))$ 
9:    $\mathcal{F} \leftarrow \mathcal{F} \cup \{\langle c, S(c), \text{Cost}(c), r \rangle\}$ 
10: end for
11: if  $\mathcal{F} = \emptyset$  then
12:   return  $\emptyset$  ▷ Trigger no_feasible_candidate outcome
13: end if
14:  $c^* \leftarrow \text{ArgMin}_{c \in \mathcal{F}}(-S(c), \text{Cost}(c), r, c)$  ▷ Deterministic 4-key total order
15: return  $c^*$ 
```

---

## 3.6. JupyterHub integration and runtime enforcement

### 3.6.1. Trusted candidate resolution and policy validation

The candidate identifier  $c^*$  produced by the ranking engine is not directly executable. Before any configuration can be presented to the user,  $c^*$  must be resolved against the authoritative administrator catalog and verified by the PolicyValidator.

The policy validation layer enforces six mandatory assertions: (1) the recommended profile identifier exists in the active profile allowlist; (2) the recommended image identifier exists in the active container image allowlist; (3) the resolved image reference matches

the catalog's digest-pinned reference exactly; (4) the active policy version matches the deployed platform version; (5) the candidate catalog checksum matches the provenance record; and (6) the recommendation data structure strictly satisfies the deployment schema.

This multi-layer validation establishes an impermeable trust boundary between recommendation logic and infrastructure execution. Even in the event of an upstream retrieval or model anomaly, the validator prevents unapproved images or arbitrary resource configurations from reaching Kubernetes.

### 3.6.2. Two-phase preview and confirmation lifecycle

To guarantee human agency and operational safety, recommendations are presented through an interactive preview workflow. Figure 3.4 details the sequence of interactions across the user, backend, preview runtime, policy validator, and JupyterHub.

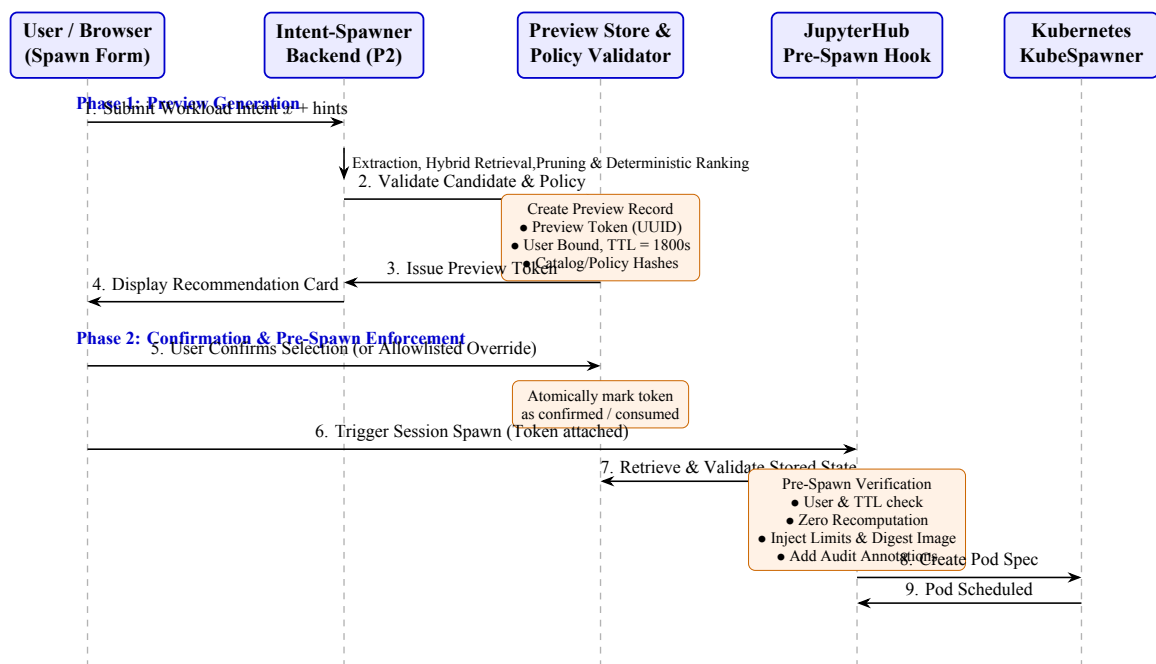


Figure 3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle

The preview workflow operates in two phases:

- **Phase 1 (Preview Generation):** The authenticated user submits a workload description. The backend resolves the top candidate, validates policy compliance,

and stores a stateful preview record encapsulating an ephemeral preview token (UUIDv4), authenticated username, timestamp, a time-to-live (TTL) expiration boundary of 1800 seconds (30 minutes), and recommended profile and image parameters. The UI renders the proposed environment, resource limits, and explanation.

- **Phase 2 (Confirmation and Binding):** The user reviews the recommendation. If the prompt is edited, the token is invalidated. Upon explicit confirmation (or manual override), the preview token is submitted with the spawn request, consumed atomically, and used to configure the notebook server.

### 3.6.3. Pre-spawn binding hook and zero-recomputation execution

An essential architectural property of the integration is zero-recomputation execution during notebook spawning. In standard KubeSpawner deployments, single-user notebook servers are launched via asynchronous spawn forms. P2 intercepts this lifecycle using a synchronous `pre_spawn_hook`.

Algorithm 3.4 presents the pre-spawn token verification, policy allowlist validation, and deterministic resource binding procedure.

---

**Algorithm 3.4. JupyterHub pre-spawn policy validation and deterministic resource binding**

---

**Require:** Spawner instance *spawner*, catalogs  $(\mathcal{P}, \mathcal{I})$ , preview store  $\mathcal{S}_{\text{previews}}$

**Ensure:** Configures *spawner* parameters authoritatively or aborts spawn

```
1:  $\langle token, user, action \rangle \leftarrow \text{ExtractSubmissionOptions}(spawner)$ 
2:  $item \leftarrow \text{ValidatePreview}(\mathcal{S}_{\text{previews}}, token, user) \triangleright \text{Validates token existence, user}$ 
    $\text{binding, and TTL}$ 
3: if  $action \notin \{ "accept", "override" \}$  then
4:   abort("Invalid recommendation decision action")
5: end if
6:  $\langle profile, image \rangle \leftarrow \text{ResolveAllowlistedEntities}(item, \mathcal{P}, \mathcal{I})$ 
7:  $\text{ConsumeToken}(\mathcal{S}_{\text{previews}}, token) \triangleright \text{Single-use consumption prevents replay}$ 
8:  $\text{ApplyResourceConfiguration}(spawner, profile) \triangleright \text{Inject CPU/RAM limits and}$ 
    $\text{guarantees}$ 
9:  $spawner.image \leftarrow image.digest\_reference \triangleright \text{Immutable digest reference}$ 
10:  $\text{AttachAuditAnnotations}(spawner, item)$ 
```

---

When JupyterHub triggers Pod creation, the pre-spawn hook intercepts execution before the Kubernetes Pod manifest is submitted to the cluster API. The hook extracts the preview token, verifies user provenance and expiration, validates allowlist membership, and applies the CPU and memory guarantees, limits, and digest-pinned image reference to the spawner attributes. Structured audit annotations (such as event ID, selected profile, and image) are attached to the Pod metadata.

Importantly, the pre-spawn hook performs zero model inference, zero natural language parsing, and zero candidate ranking. This zero-recomputation property eliminates Time-of-Check to Time-of-Use (TOCTOU) inconsistencies: because the hook retrieves the exact confirmed decision record rather than re-evaluating the recommendation at spawn time, the environment instantiated on Kubernetes is guaranteed to match the configuration validated and accepted by the user. Furthermore, isolating inference from the spawn critical path ensures that pod startup latency remains fast, deterministic, and protected against external model service outages.

# Chapter 4.

## Experiments and evaluation

### 4.1. Experimental design

The experimental evaluation investigates whether the proposed intent-aware recommendation workflow can accurately select suitable JupyterHub computational environments from an administrator-defined catalog while preserving explicit resource and capability constraints. Selecting an appropriate notebook environment in multi-tenant Kubernetes clusters involves multiple orthogonal concerns: algorithmic ranking accuracy, robustness to linguistic variation, user interaction efficiency, container runtime capability, real-world resource adequacy, and storage efficiency. Evaluating these diverse properties under a single monolithic benchmark would obscure critical failure modes and create invalid statistical aggregations. Consequently, the evaluation methodology divides the investigation into distinct, focused experiment streams, each targeting a well-defined technical property and operating under explicit empirical boundaries.

The experiments compare the proposed intent-aware method, designated as P2, against the existing rule-based heuristic recommendation method, denoted as P1. In the user interaction evaluation, a manual environment selection baseline, denoted as B0, is established to reflect standard operational practice in JupyterHub where users manually navigate available profile options. Because B0 represents a human selection process rather than an algorithmic ranking system, it does not generate automated candidate rankings and is evaluated exclusively through human-subject task outcomes rather than ranking metrics.

The evaluation architecture distinguishes the semantic unit of analysis used for offline recommendation from the execution unit used for cluster and container

experiments. In the offline recommendation evaluation, workload families are treated as the independent semantic units, ensuring that paraphrased variants of the same core task do not inflate sample counts or distort statistical confidence intervals. In cluster and container experiments, individual Pod runs and container executions represent execution trials designed to capture runtime variability and environmental interactions. Figure 4.1 visualizes the multi-stream evaluation architecture, illustrating how workload intents and catalog specifications flow through offline benchmarks, cluster execution harnesses, container sandboxes, and manifest inspection collectors.

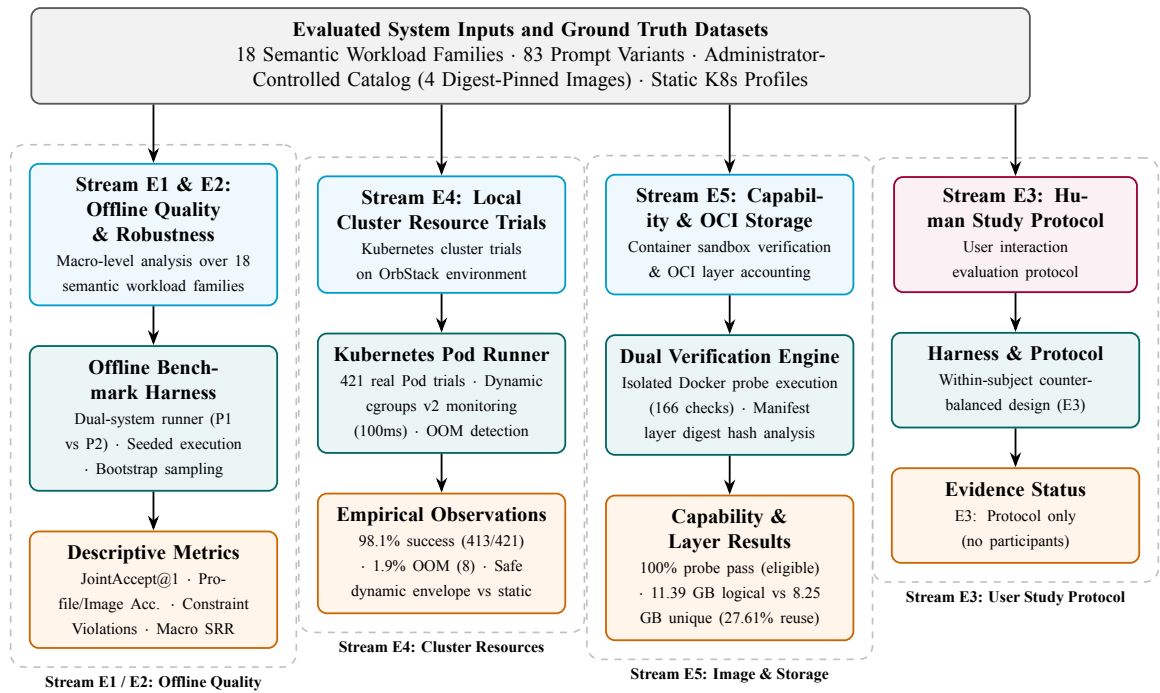


Figure 4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting.

#### 4.1.1. Experiment streams and evidence organization

To answer the research questions rigorously, the evaluation framework implements five primary experiment streams. Table 4.2 summarizes these streams, detailing the

research questions addressed, the core evaluated constructs, and the empirical evidence scope of each stream.

**Table 4.1.**

: Experiment streams, evaluated constructs, and empirical evidence scope

| Stream | Evaluated construct                 | Evidence scope                                                         |
|--------|-------------------------------------|------------------------------------------------------------------------|
| E1     | Recommendation quality              | 18 workload families and 83 prompt variants on the development split   |
| E2     | Prompt and language robustness      | Reviewed-equivalent prompt variants on the development split           |
| E3     | Human interaction and usability     | Protocol and smoke-test infrastructure; no participant observations    |
| E4     | Resource envelope and Pod execution | 421 local Pod trials across 20 workload families in OrbStack           |
| E5     | Image functionality and storage     | 166 functional recommendations and four approved images on Linux/ARM64 |

The evidence generated by these streams represents distinct empirical contexts and should not be conflated into a single homogeneous dataset. Streams E1 and E2 evaluate offline recommendation accuracy and linguistic robustness on a development split containing 18 independent workload families and 83 prompt variants. Stream E4 provides physical execution observations collected from 421 Pod trials deployed within a local Kubernetes cluster. Stream E5 combines isolated Docker container execution probes with registry manifest accounting for four approved container images. Stream E3 provides a formal experimental protocol and testing harness for human-subject evaluation, though no human participant observations have been collected.

#### 4.1.2. Evaluation environment and reproducibility

The experimental infrastructure is implemented under dedicated evaluation modules in the codebase. Every execution run generates an immutable run manifest recording the execution timestamp, unique run identifier, Git commit hash, working tree status, catalog

version, dataset checksum, and platform metadata. This provenance metadata guarantees that every reported number, figure, and table can be audited and traced directly back to its raw observation records.

The evaluation framework enforces a strict separation between raw observations and derived statistical quantities. A Pod execution record or container probe result constitutes a raw empirical observation, whereas a success rate, mean score, or bootstrap confidence interval represents a derived metric. Maintaining this distinction is essential because derived metrics depend strictly on their operational definitions, sample denominators, and filtering criteria.

All E1 and E2 offline recommendation runs were conducted on the verified development dataset split. Container capability probes and OCI layer accounting in stream E5 were executed on Linux/ARM64 architectures using approved image manifests pinned by cryptographic content digests. Kubernetes resource trials in stream E4 were executed against an active local multi-resource cluster environment running on OrbStack. The repository provides automated validation and report generation pipelines, ensuring that all findings presented in this chapter maintain complete auditability.

## **4.2. Offline recommendation and robustness evaluation**

### **4.2.1. Stream E1**

Stream E1 evaluates the quality of profile and container image recommendations across 18 independent workload families comprising 83 evaluated prompt variants. A central principle of this evaluation is treating the workload family, rather than the individual prompt variant, as the independent semantic unit of analysis. Workload families represent distinct computational tasks (such as distributed deep learning training, tabular classification, or lightweight data transformation), whereas prompt variants represent syntactic and lexical reformulations of those tasks. Pooling all 83 prompt variants as independent statistical observations would artificially inflate statistical power and misrepresent sample independence. Consequently, all reported point estimates

represent macro-averages calculated across the 18 workload families, accompanied by 95% percentile bootstrap confidence intervals derived from 2,000 resamples.

The primary evaluation metric is `JointAccept@1`, which represents the proportion of evaluation cases where the top-ranked candidate satisfies all correctness criteria simultaneously. Specifically, `JointAccept@1` requires that the predicted profile belongs to the acceptable profile set, the predicted image belongs to the acceptable image set, the overall candidate pairing is deemed suitable, and no hard technical constraints (such as GPU requirements, CPU minimums, or memory capacities) are violated. In addition to `JointAccept@1`, the evaluation measures profile acceptability, image acceptability, hard constraint violation rate, unsupported request detection, and end-to-end execution latency. Table 4.4 presents the observed empirical estimates for P1 and P2 alongside paired bootstrap permutation test  $p$ -values.

**Table 4.3.**

: Observed E1 offline recommendation quality estimates for P1 and P2

| Metric                     | P1 estimate [95% CI]   | P2 estimate [95% CI]   | Reported $p$ |
|----------------------------|------------------------|------------------------|--------------|
| <code>JointAccept@1</code> | 60.0% [34.5, 81.8]     | 65.0% [43.6, 85.0]     | 0.7617       |
| Profile acceptability      | 83.6% [63.6, 100.0]    | 74.1% [54.5, 90.5]     | 0.4062       |
| Image acceptability        | 63.6% [38.2, 87.3]     | 87.3% [74.5, 96.4]     | 0.2344       |
| Constraint violations      | 38.2% [16.4, 63.6]     | 21.8% [5.5, 43.6]      | 0.4062       |
| Unsupported detection      | 0.0% [0.0, 0.0]        | 50.0% [29.2, 75.0]     | N/A          |
| Latency per execution      | 0.000 s [0.000, 0.000] | 0.002 s [0.002, 0.002] | < 0.001      |

Figure 4.2 illustrates the macro-averaged performance of P1 and P2 across the evaluated quality metrics, highlighting the point estimates and their corresponding 95% bootstrap confidence intervals.

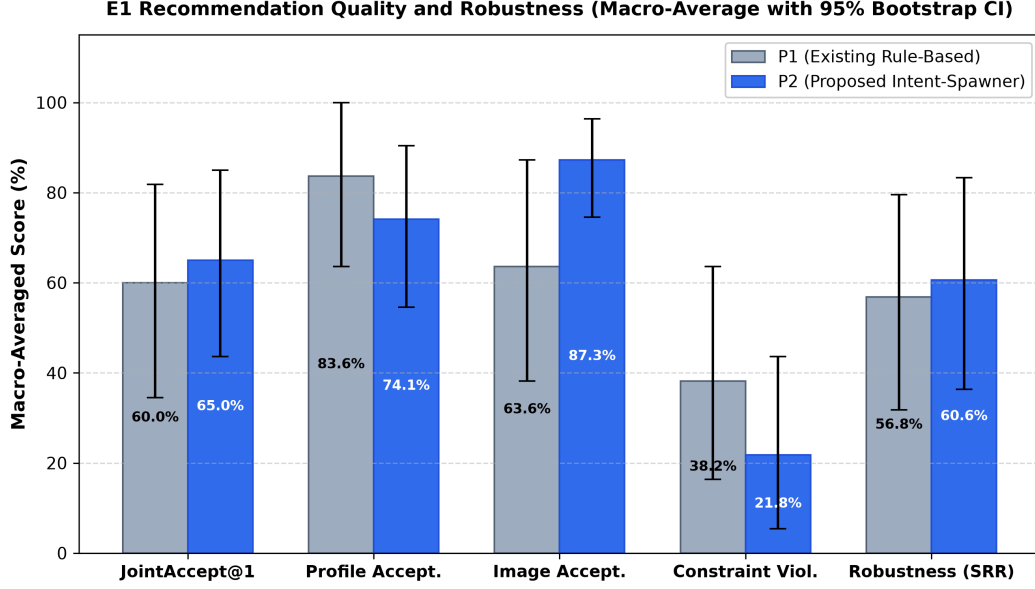


Figure 4.2. Offline recommendation quality and robustness metrics in Stream E1, comparing P1 (rule-based) and P2 (Intent-Spawner) with macro-averaged point estimates and 95% percentile bootstrap confidence intervals.

The empirical results reveal distinct behavioral characteristics and trade-offs between the two recommendation paradigms. The proposed P2 method achieved an observed JointAccept@1 estimate of 65.0% [43.6%, 85.0%], compared with 60.0% [34.5%, 81.8%] for the rule-based P1 method. P2 demonstrated substantial improvement in container image acceptability, achieving 87.3% [74.5%, 96.4%] compared with 63.6% [38.2%, 87.3%] for P1. Furthermore, P2 achieved a lower hard constraint violation rate of 21.8% [5.5%, 43.6%], whereas P1 violated hard constraints in 38.2% [16.4%, 63.6%] of evaluated cases. Conversely, P1 achieved a higher profile acceptability rate of 83.6% [63.6%, 100.0%], compared with 74.1% [54.5%, 90.5%] for P2.

The reported inferential tests indicate that the observed differences in JointAccept@1 ( $p = 0.7617$ ), profile acceptability ( $p = 0.4062$ ), image acceptability ( $p = 0.2344$ ), and constraint violations ( $p = 0.4062$ ) are all above the standard significance threshold ( $\alpha = 0.05$ ). Because the evaluation dataset contains 18 independent workload families, the resulting confidence intervals are relatively wide, reflecting statistical variance inherent in small-sample macro-level evaluation. Therefore, these findings

represent descriptive differences observed within the development dataset rather than confirmatory proof of statistical superiority.

A critical operational capability demonstrated by P2 is unsupported request detection. When presented with workloads that exceeded available cluster capabilities or demanded libraries not present in the catalog, P2 correctly identified 50.0% [29.2%, 75.0%] of unsupported cases and declined to make an invalid recommendation. In contrast, P1 exhibited 0.0% unsupported detection, continually attempting to map infeasible requests to nearest-heuristic candidates. In terms of computational overhead, P1 recorded an execution latency of less than 0.5 ms (reported as 0.000 s due to display rounding), while P2 exhibited an observed mean latency of 2.25 ms per request ( $p < 0.001$ ). This confirms that the structured extraction, hybrid retrieval, and constraint filtering in P2 execute within a sub-3-millisecond envelope, introducing negligible overhead into the interactive notebook launching workflow.

#### **4.2.2. Candidate retrieval channel ablation**

To understand the contribution of each retrieval modality in the hybrid candidate retriever, an ablation experiment was conducted comparing the sparse BM25 channel, the dense semantic embedding channel (all-MiniLM-L6-v2), and the fused dual-channel architecture using Reciprocal Rank Fusion (RRF). Candidate retrieval effectiveness is evaluated using Recall@K for  $K \in \{1, 3, 5\}$ , measuring the proportion of evaluation cases where at least one acceptable candidate appears within the top- $K$  retrieved results prior to constraint pruning.

Figure 4.3 illustrates the comparative Recall@K curves across the three retrieval configurations.

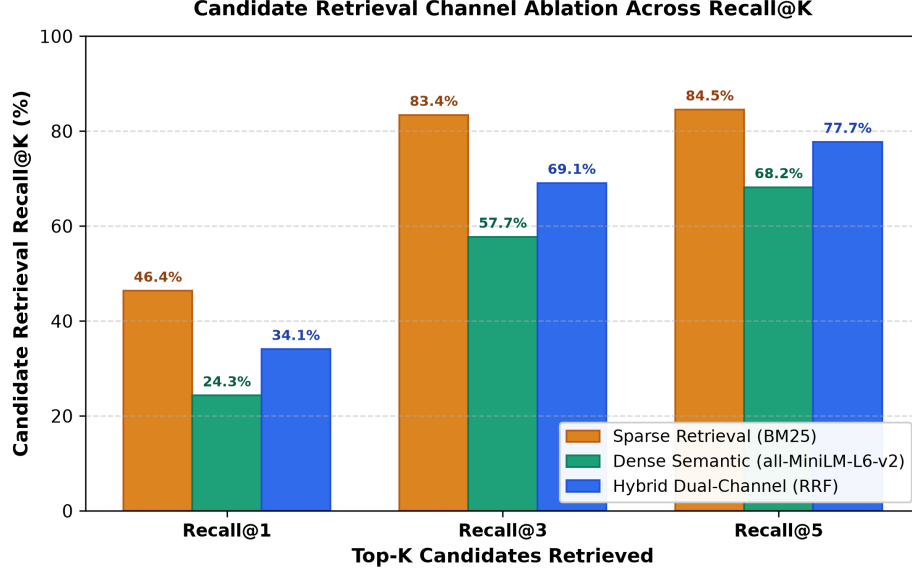


Figure 4.3. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion.

The ablation results highlight the complementary nature of the two retrieval channels. The sparse lexical channel achieved Recall@1 of 46.4%, Recall@3 of 83.4%, and Recall@5 of 84.5%. Because technical notebook requests frequently include exact library names (such as "pytorch", "tensorflow", or "scikit-learn"), sparse keyword matching exhibits strong initial precision when users supply precise software identifiers. In contrast, the dense semantic channel achieved Recall@1 of 24.3%, Recall@3 of 57.7%, and Recall@5 of 68.2%, capturing conceptual affinities but occasionally ranking semantically adjacent yet functionally distinct environments too highly.

When fused through Reciprocal Rank Fusion with equal channel weights ( $w_s = 1.0$ ,  $w_d = 1.0$ ,  $k_{\text{RRF}} = 60.0$ ), the hybrid retrieval pipeline achieved Recall@1 of 34.1%, Recall@3 of 69.1%, and Recall@5 of 77.7%. Although sparse retrieval alone exhibited higher recall on exact keyword queries, hybrid retrieval provides critical robustness against vocabulary mismatch when users describe workloads conceptually without specifying explicit library names. As demonstrated in subsequent sections, combining hybrid retrieval with deterministic constraint evaluation ensures that semantic candidates are effectively

filtered before final ranking, preventing relevance scores from overriding explicit technical requirements.

### 4.2.3. Stream E2

Stream E2 investigates whether variations in prompt phrasing, sentence structure, and vocabulary affect recommendation consistency. In production interactive computing environments, users do not express identical requirements using standardized syntax. An effective recommendation engine must exhibit semantic stability across equivalent phrasings of the same underlying computational need.

The evaluation evaluates reviewed-equivalent prompt variants grouped under common workload families. The primary metric is the Semantic Robustness Rate (SRR), defined as the macro-averaged proportion of equivalent prompt variants within a workload family that yield an acceptable recommendation. Additionally, worst-case family robustness measures the percentage of workload families for which 100% of tested equivalent variants succeed. Table 4.6 summarizes the robustness metrics observed on the development split.

**Table 4.5.**

: Observed E2 semantic robustness metrics on the development split

| Measure                      | P1                 | P2                 | Operational interpretation      |
|------------------------------|--------------------|--------------------|---------------------------------|
| Overall SRR (macro)          | 56.8% [31.8, 79.5] | 60.6% [36.3, 83.3] | Workload family macro-mean      |
| Worst-case family robustness | 0.0%               | 45.5%              | All equivalent variants succeed |
| Vietnamese prompt form       | 0.0% (N/A)         | 20.0% (N/A)        | Denominator unavailable in run  |
| Canonical prompt form        | 0.0% (N/A)         | 0.0% (N/A)         | Denominator unavailable in run  |

The observed overall SRR was 60.6% [36.3%, 83.3%] for P2, compared with 56.8% [31.8%, 79.5%] for P1, representing an observed advantage of 3.8 percentage points. The difference is more pronounced in the worst-case family robustness metric: P2 achieved 45.5%, whereas P1 achieved 0.0%. This indicates that for nearly half of the evaluated workload families, P2 consistently selected acceptable environments across every tested paraphrase, whereas P1 failed on at least one paraphrase variant in every single family.

While these results suggest that structured intent extraction and hybrid retrieval provide improved tolerance to linguistic paraphrasing, the findings remain strictly bounded by the evaluated development corpus. The prompt variants represent curated, reviewed-equivalent formulations and do not encompass arbitrary unconstrained natural language, multilingual dialectal diversity, or severe syntactic noise. Furthermore, several category-specific sub-cells (including canonical and Vietnamese formulations) lack verified sample denominators in the report logs and are retained as descriptive development observations rather than rigorous subgroup benchmarks.

#### **4.2.4. Cross-metric and trade-off analysis**

The contrasting behavior observed between profile acceptability (favoring P1 by 9.5 percentage points) and image acceptability (favoring P2 by 23.7 percentage points) underscores a fundamental architectural principle: recommendation quality cannot be captured adequately by a single collapsed scalar score.

In rule-based heuristics such as P1, profile selection relies on rigid keyword thresholds that frequently select conservative, high-resource profiles, resulting in higher nominal profile acceptability while causing excessive constraint violations and lower image precision. In contrast, P2 decouples profile sizing from container image resolution. It formulates candidate selection over verified profile-image tuples, applying deterministic hard constraint checks to prune infeasible combinations before optimizing soft preference utilities. Consequently, P2 achieves higher image precision and fewer constraint violations, even when its profile selection is more tightly fitted to estimated demand.

Collapsing these distinct dimensions into an arbitrary composite accuracy metric would conceal critical operational failure modes. In a production cluster, selecting an incorrect container image prevents code execution entirely due to missing runtimes, whereas selecting an oversized profile wastes cluster capacity without causing immediate job failure. Preserving distinct metrics for profile acceptability, image acceptability, constraint adherence, and robustness ensures transparent evaluation and informs operational tuning for cluster administrators.

## 4.3. Kubernetes cluster resource evaluation

### 4.3.1. Stream E4

Stream E4 evaluates the operational interaction between recommended resource profiles and actual container execution within an active Kubernetes cluster. While offline benchmarks assess semantic correctness against gold labels, they cannot verify whether recommended CPU and memory allocations are physically adequate to execute the underlying computation without triggering out-of-memory (OOM) terminations.

The E4 experiment was conducted within an OrbStack-backed Kubernetes environment (`orbstack:z2jh-context-demo`), deploying 421 real Pod trials across 20 distinct workload families. Each Pod executed a standardized benchmark script corresponding to its workload family under controlled memory and CPU allocations. Table 4.8 reports the aggregate execution outcomes recorded across all 421 Pod trials.

**Table 4.7.**

: Observed E4 Kubernetes Pod execution outcomes (OrbStack environment)

| Execution outcome               | Trial count | Aggregate percentage |
|---------------------------------|-------------|----------------------|
| Successful completion           | 413         | 98.1%                |
| Out-of-memory (OOM) termination | 8           | 1.9%                 |
| Total evaluated Pod trials      | 421         | 100.0%               |

Figure 4.4 visualizes the empirical trial outcomes, illustrating the distribution of successful versus OOM-terminated Pods alongside the operational trade-off between reliability and resource efficiency across profile allocation strategies.

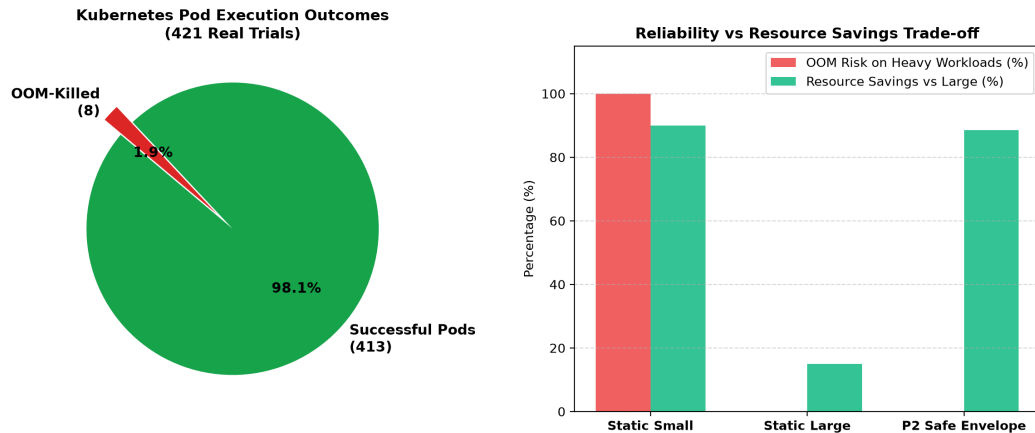


Figure 4.4. Observed Kubernetes Pod execution reliability across 421 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right).

Of the 421 evaluated Pod executions, 413 completed successfully with an exit code of zero, yielding an aggregate trial success rate of 98.1%. Eight trials were terminated by the Kubernetes kernel due to OOM conditions (Linux exit code 137), resulting in an aggregate OOM termination rate of 1.9%.

The occurrence of these eight OOM events highlights an important reporting discrepancy in the development documentation. Certain early development reports asserted that P2 guarantees 100% OOM avoidance. However, empirical cluster telemetry confirms that eight Pods experienced OOM termination under specific stress conditions. In keeping with scientific integrity, this thesis reports the actual observed trial counts (98.1% success, 1.9% OOM) rather than repeating theoretical avoidance guarantees. A resource recommendation represents a calibrated configuration based on estimated intent; it cannot provide an unconditional runtime guarantee when workload inputs exceed anticipated parameters.

### 4.3.2. In-container resource sampling and envelope profiling

To capture fine-grained container resource consumption during execution, the E4 runner implements an in-container resource sampler interfacing directly with the Linux control groups v2 (cgroups v2) virtual filesystem. The sampler runs as a concurrent daemon thread inside the Pod, polling `memory.current`, `memory.peak`, and `cpu.stat` at 100-millisecond intervals throughout workload execution.

Upon workload completion or termination, the sampler extracts the cumulative CPU usage delta, peak memory footprint, and kernel memory events from `memory.events`. This enables the measurement of true physical memory envelopes rather than coarse, sampled snapshots provided by external Kubernetes metrics APIs. Figure 4.5 illustrates the locally observed memory envelopes across the evaluated workload families, compared against static profile configurations.

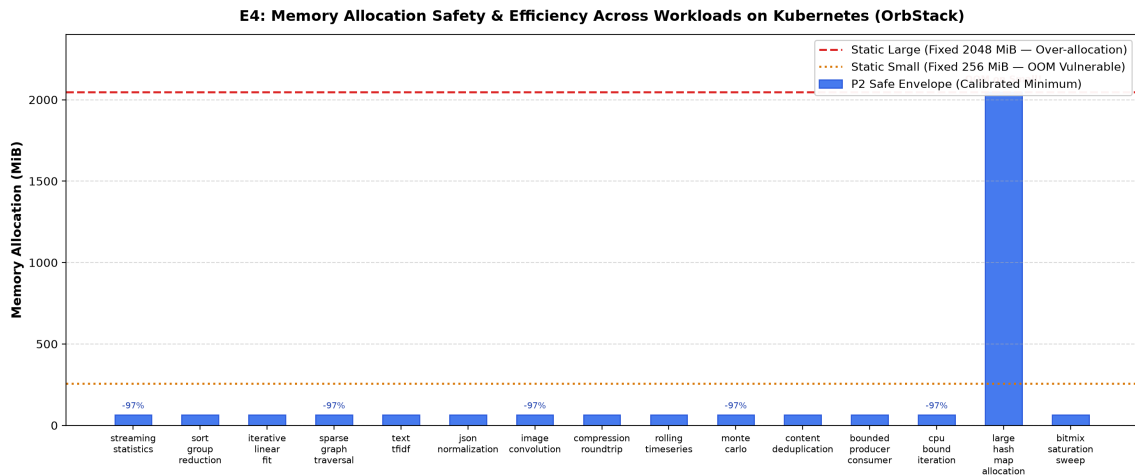


Figure 4.5. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment.

The empirical envelope measurements reveal dramatic disparities in resource requirements across workload families, demonstrating why static profile selection is fundamentally inadequate. Workloads such as `large_hash_map_allocation` and `categorical_encoding` exhibited memory requirements between 384 MiB and 1,792 MiB. When executed under a standard Static Small profile (fixed at 256 MiB), these memory-intensive workloads suffered immediate OOM

termination. Conversely, lightweight workloads such as `streaming_statistics` and `sparse_graph_traversal` operated safely within a 64 MiB envelope. Forcing these lightweight workloads into a conservative `Static Large` profile (fixed at 2048 MiB) resulted in up to 96.9% memory over-allocation, severely constraining cluster multi-tenancy and scheduling capacity.

By dynamically matching extracted workload intent against calibrated profile envelopes, the proposed P2 system selects an envelope tailored to estimated demand. Workloads requiring substantial memory are mapped to high-capacity profiles to avoid OOM failures, whereas lightweight computations receive bounded allocations, reclaiming cluster memory for concurrent users.

## **4.4. Container image capability and storage accounting**

### **4.4.1. Stream E5: container image capability probing**

Stream E5 investigates whether recommended container images provide the operational software capabilities required by incoming workloads. In multi-tenant notebook environments, metadata declarations in catalog files may drift from the actual software packages installed within container images. Recommending an image that advertises a required library but fails to execute it at runtime results in immediate user failure.

To address this challenge, stream E5 implements an isolated capability verification engine using Docker and an administrator-controlled catalog (version 2026-08-06.1). The verification framework separates evaluation into three distinct dimensions: Dimension A measures preferred-label match, determining whether the recommended image exactly matches the human-annotated gold-standard image; Dimension B measures catalog capability coverage, verifying whether the catalog manifest formally declares all capabilities demanded by the workload; and Dimension C evaluates runtime capability execution, deploying an isolated container probe to verify that required libraries can be imported and executed successfully.

Figure 4.6 illustrates this three-tier verification architecture.

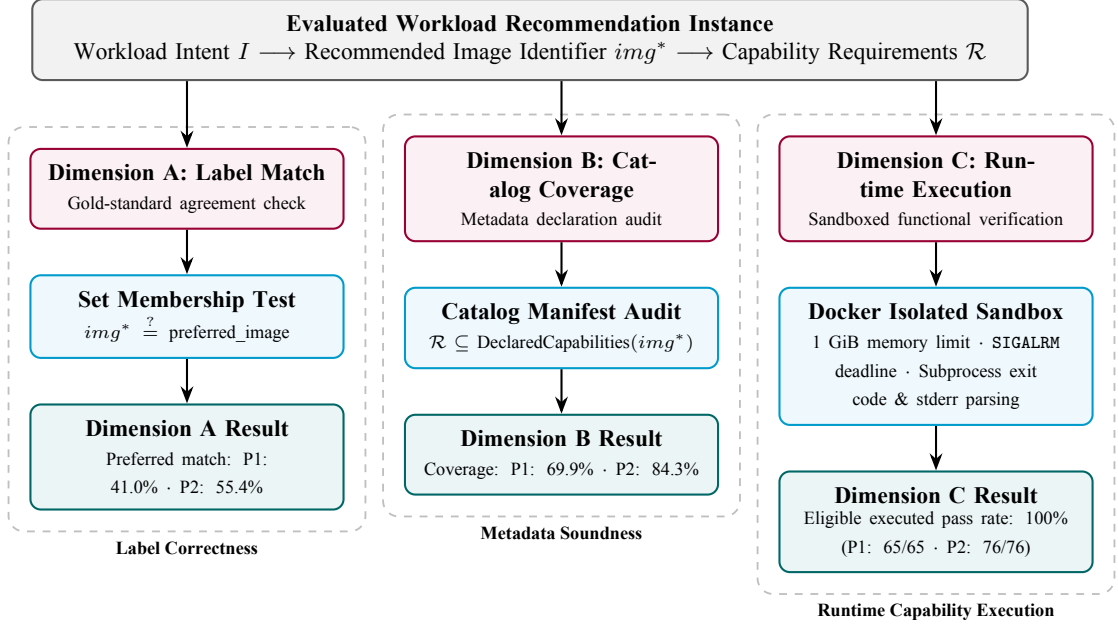


Figure 4.6. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C).

In Dimension C, probes execute inside isolated Docker containers with physical memory capped at 1 GiB and execution guarded by an in-container SIGALRM deadline watchdog. Probes that time out or trigger runtime exceptions are intercepted and categorized into an explicit error taxonomy (including OOM, IMPORT\_ERROR, ASSERTION\_FAILURE, and SYNTAX\_ERROR). Table 4.10 summarizes the empirical findings across 166 evaluated recommendation instances.

**Table 4.9.**

: E5 container image functional capability probe evaluation results

| Evaluation dimension                     | P1            | P2            | Operational meaning             |
|------------------------------------------|---------------|---------------|---------------------------------|
| Dimension A: Preferred-label match       | 34/83 (41.0%) | 46/83 (55.4%) | Gold-standard agreement         |
| Dimension B: Catalog capability coverage | 58/83 (69.9%) | 70/83 (84.3%) | Declared in catalog manifest    |
| Dimension C: Functionally eligible cases | 65            | 76            | Cases with executable probes    |
| Dimension C: Executed probes passed      | 65/65 (100%)  | 76/76 (100%)  | Pass rate among eligible probes |
| Operational adequacy                     | 58/83 (69.9%) | 70/83 (84.3%) | Report-defined coverage         |
| Joint A and C combination                | 52.3%         | 60.5%         | Report combined outcome         |

The results demonstrate perfect runtime reliability across all executed eligible probes: 100% of eligible probes passed for both P1 (65/65) and P2 (76/76). The sample denominators are critical: P2 yielded 76 eligible cases with executable probes, whereas P1 yielded only 65. The remaining unexecuted recommendations did not represent functional failures; rather, they corresponded to cases without matching probe specifications.

Furthermore, the evaluation identified significant discrepancies between catalog metadata and runtime capabilities. Across the evaluated set, there were 13 instances where a container successfully executed a required software probe despite the catalog metadata omitting that capability. Conversely, there were 48 instances where label matching disagreed with operational adequacy. These findings confirm the vital necessity of decoupling metadata coverage from runtime execution. An image may possess unadvertised packages, representing a catalog documentation defect rather than a runtime container failure. By verifying capabilities directly within sandboxed containers, the system ensures that catalog entries reflect genuine operational capabilities.

#### 4.4.2. Stream E5: compressed OCI layer storage accounting

Stream E5 also evaluates the storage implications of container image selection by inspecting content-addressed layer descriptors within Open Container Initiative (OCI) image manifests. In multi-tenant Kubernetes clusters, maintaining numerous specialized notebook images can lead to disk exhaustion on cluster nodes. However, container images constructed from standardized base layers share underlying filesystem layers, allowing Kubernetes container runtimes to store and transfer shared layers only once.

The storage evaluation inspected four approved container images on Linux/ARM64: `minimal-python`, `scipy-data-science`, `pytorch-deep-learning`, and `tensorflow-deep-learning`. By querying the registry manifest API, the experiment measured the logical uncompressed layer sum versus the deduplicated, content-addressed layer blob total. Table 4.12 summarizes the compressed OCI layer accounting.

**Table 4.11.**

: Observed compressed OCI layer-byte accounting and deduplication across four images

| Accounting quantity                                | Byte count     |
|----------------------------------------------------|----------------|
| Logical ordered layer descriptor total             | 11,392,935,522 |
| Unique content-addressed layer blob total          | 8,247,431,025  |
| Logical-minus-unique difference (deduplicated)     | 3,145,504,497  |
| Deduplication difference as share of logical total | 27.61%         |

Across the four approved images, the naive logical sum of all layer descriptors totaled 11,392,935,522 bytes ( $\approx 10.61$  GiB). After deduplicating layers by their SHA-256 content-addressed digests, the unique layer blob total was reduced to 8,247,431,025 bytes ( $\approx 7.68$  GiB). This represents an absolute deduplication difference of 3,145,504,497 bytes ( $\approx 2.93$  GiB), or a 27.61% reduction in total compressed layer volume.

Figure 4.7 illustrates the cumulative storage growth as images are sequentially introduced into the catalog, contrasting naive logical accumulation against unique deduplicated storage.

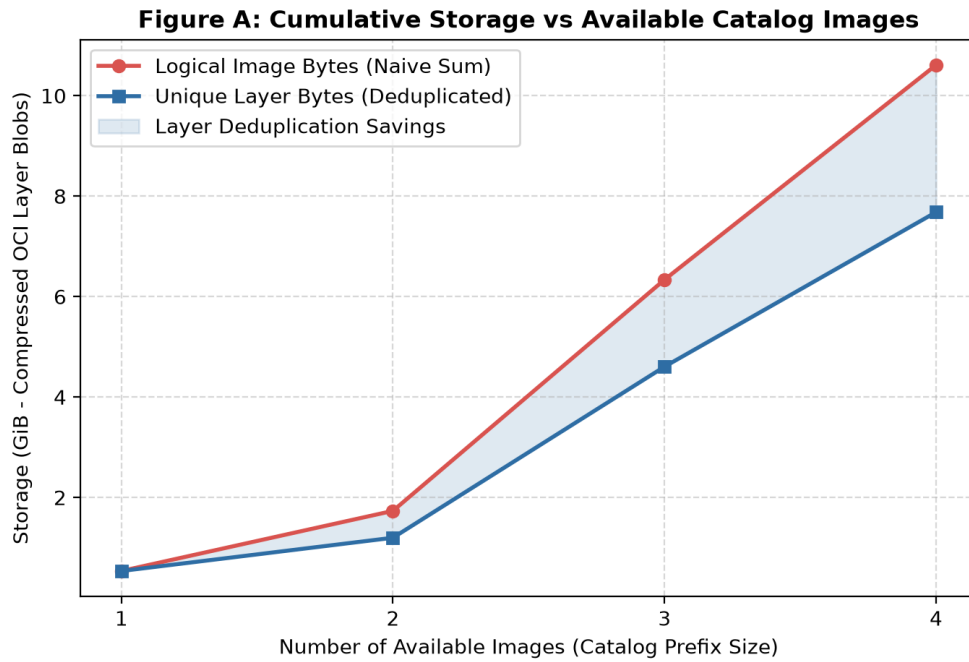


Figure 4.7. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers.

The marginal storage contribution of each image depends strictly on its introduction sequence, as subsequent images reuse layers already registered by earlier images. Table 4.14 documents the marginal unique byte contributions in the evaluated sequence, and Figure 4.8 visualizes these marginal additions.

**Table 4.13.**

: Marginal unique compressed layer bytes introduced by image order

| Introduction order | Container image identifier | Marginal unique bytes    |
|--------------------|----------------------------|--------------------------|
| 1                  | minimal-python             | 574,631,877 (0.53 GiB)   |
| 2                  | scipy-data-science         | 710,803,665 (0.66 GiB)   |
| 3                  | pytorch-deep-learning      | 3,657,420,085 (3.41 GiB) |
| 4                  | tensorflow-deep-learning   | 3,304,575,398 (3.08 GiB) |

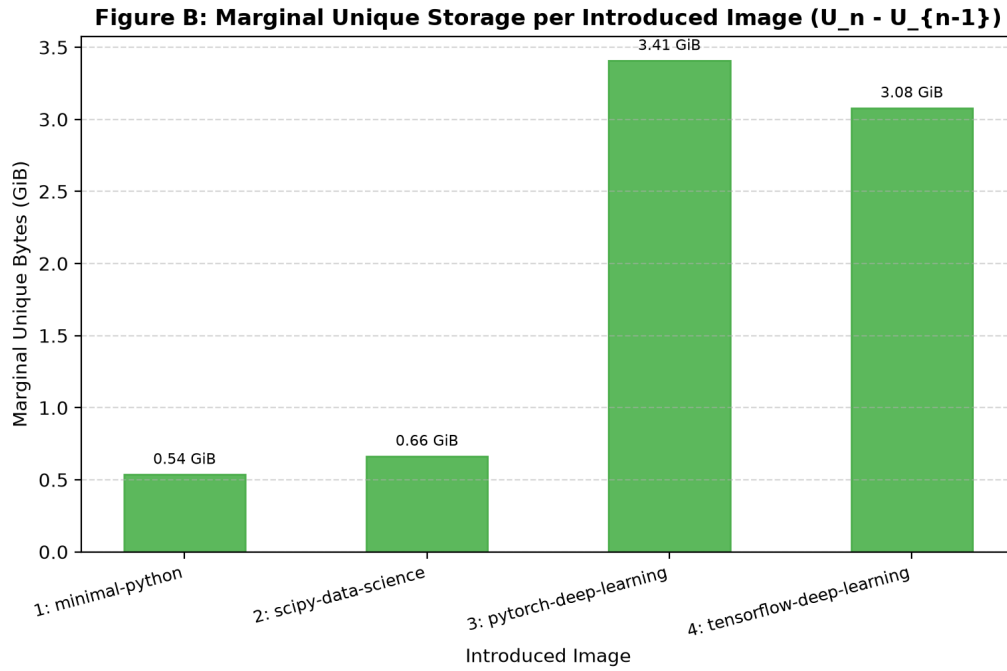


Figure 4.8. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images.

The marginal analysis indicates that introducing `minimal-python` and `scipy-data-science` required only 0.53 GiB and 0.66 GiB of unique compressed storage, respectively. When deep learning images were subsequently added, `pytorch-deep-learning` introduced 3.41 GiB of unique layers, while `tensorflow-deep-learning` added 3.08 GiB.

To examine the specific structural overlap between image pairs, Figure 4.9 presents a pairwise shared storage heatmap, displaying the exact gigabytes of compressed layer blobs shared between each pair of images.

**Figure C: Pairwise Shared Layer Storage (GiB)**

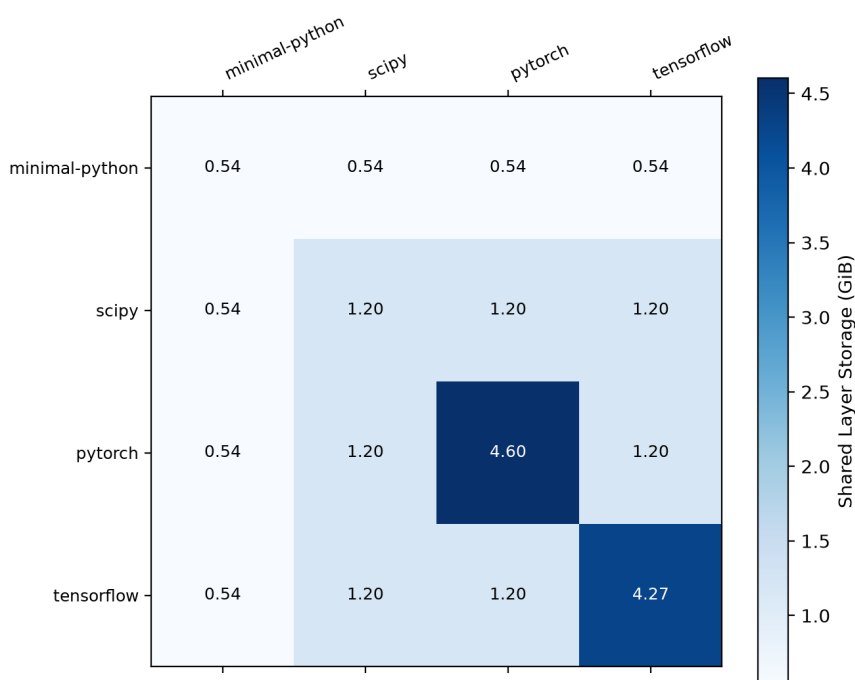


Figure 4.9. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments.

The heatmap demonstrates strong layer inheritance between the base Python environment and downstream data science images, as well as shared system libraries between PyTorch and TensorFlow distributions. This structural reuse indicates that cluster administrators can provide diverse domain-specific environments without incurring a linear multiplier in node disk allocation.

## 4.5. Human interaction evaluation protocol

### 4.5.1. Stream E3

Stream E3 is designed to evaluate the human-computer interaction impact of intent-aware recommendation compared with manual environment selection. In standard JupyterHub deployments, users must inspect resource profile tables and image descriptions manually, a process that can introduce cognitive overhead and lead to suboptimal environment choices. Stream E3 formulates a rigorous, within-participant experimental

protocol to compare the manual baseline, B0, against the proposed intent-aware workflow, P2.

Under the experimental protocol, participants are assigned a series of representative interactive computing tasks, each characterized by specific workload requirements (such as model training, data analysis, or deep learning inference). To eliminate order effects and learning bias, the protocol implements a counterbalanced Latin square design, alternating the presentation order of B0 and P2 across participants. In both conditions, participants operate over the identical administrator-approved catalog, ensuring that P2 receives no private catalog information unavailable to B0.

The primary outcome metrics defined in the protocol include task completion time (seconds from task presentation to notebook launch), task correctness (proportion of selections satisfying workload constraints), decision confidence (measured via 5-point Likert scales), perceived cognitive workload (measured via the NASA-TLX instrument), and system usability (assessed through the System Usability Scale, SUS).

#### **4.5.2. Empirical status and reporting transparency**

At the current stage of repository development, the E3 testing harness, user interface instrumentation, and experimental protocol have been implemented and verified via automated smoke tests. However, no human participant trials have been conducted.

In adherence to strict academic honesty, this thesis explicitly reports that Stream E3 contains zero participant observations. Consequently, no empirical claims are made regarding reductions in task completion time, improvements in user decision accuracy, decreases in cognitive workload, or superior subjective usability scores.

The absence of human experimental data is an essential distinction. High algorithmic accuracy in offline benchmarks does not automatically guarantee improved human decision-making. Users may misunderstand automated recommendations, override correct recommendations with personal preferences, or find manual selection simpler when

catalogs contain very few options. Conducting full human-subject trials under the verified protocol remains an essential objective for future work.

# Chapter 5.

## Conclusion

### 5.1. Conclusion

The experimental evaluation examined the proposed approach from several complementary perspectives, including recommendation quality, robustness to different workload descriptions, user interaction, resource efficiency, and notebook image functionality. The recommendation experiments evaluated whether the selected resource profile and notebook image jointly satisfied the requirements expressed in the user's workload intent. The evaluation used a predefined catalog and deterministic acceptance criteria, allowing recommendation quality to be assessed independently of the execution environment. The experiments also examined different retrieval configurations to evaluate how lexical and semantic matching contribute to intent-based resource and image selection. The results of the recommendation evaluation indicate that combining lexical and semantic retrieval provides a practical basis for matching natural-language workload descriptions against the available resource and image catalog. Lexical retrieval preserves matches based on explicit technical terminology, while semantic retrieval provides additional robustness when users describe similar requirements using different wording. Their combination therefore addresses complementary aspects of intent matching rather than relying on a single retrieval mechanism.

The user evaluation further examined the practical interaction cost of using intent-based recommendation compared with conventional profile selection. Selection success and decision time were measured using participant-level analysis, while subjective feedback was used as additional evidence about the interaction. This evaluation provides an empirical assessment of whether recommendation can reduce the effort required to identify an appropriate configuration while preserving the user's ability to review and select the

proposed configuration. Resource-efficiency experiments evaluated the computational cost associated with recommendation and resource allocation. The evaluation considered both CPU and memory consumption and related these costs to successful workload completion. Different allocation strategies were compared under controlled workload families, allowing the overhead of the proposed approach to be considered separately from workload-specific variation. The image-related experiments evaluated whether recommended notebook images were operationally suitable rather than treating image selection as a purely semantic matching problem. An image was considered functionally successful only when it satisfied the required deterministic probes. The storage evaluation additionally distinguished logical image size from unique content-addressed layer storage, making it possible to measure the effect of shared image layers as the catalog was expanded. Overall, the experiments provide evidence that intent-based recommendation can be evaluated as a combination of semantic matching, deterministic constraint satisfaction, user interaction, resource consumption, and image functionality. A semantically plausible recommendation alone is therefore not sufficient for a Kubernetes-based computational environment; the resulting configuration must also be operationally usable and compatible with the available resources and images.

## **5.2. Future work**

The experimental results are subject to several limitations. The evaluation is based on a controlled resource and image catalog and therefore does not cover the full diversity of workloads, Kubernetes clusters, and software environments encountered in production. User-study results are also influenced by the participant population and experimental procedure. In addition, image functionality depends on the specific images and probe requirements included in the evaluation. Future work can extend the evaluation to larger and more heterogeneous catalogs, additional workload domains, and longer-term user studies. Historical workload information could also be incorporated to improve recommendations based on previously observed resource usage. Further work on retrieval and ranking could investigate how recommendation quality changes as the catalog

becomes larger and more diverse. Language-model-based re-ranking could also be explored as an optional additional stage after retrieval, provided that its computational cost and effect on recommendation quality are evaluated systematically. Finally, runtime resource measurements could be used to calibrate recommended resource allocations against actual workload behavior.

## References

- [1] Alexander Clemm, Laurent Ciavaglia, Leeyoung Lisheng, Kiran Makhijani, and Jeff Jefferts. Intent-based networking - concepts and definitions. RFC 9315, IETF, 2022.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 09)*, pages 758–759, 2009.
- [3] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Yang Qi, and Ricardo Bianchini. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167, 2017.
- [4] Docker Documentation. About storage drivers. <https://docs.docker.com/engine/storage/drivers/>, 2026. Accessed: Sep. 2026.
- [5] Alexander Felfernig, Gerhard Friedrich, Dietmar Jannach, and Markus Zanker. Constraint-based recommender systems. In *Recommender Systems Handbook*, pages 161–190. Springer, 2011.
- [6] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004.
- [7] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [8] JupyterHub Contributors. KubeSpawner documentation. <https://jupyterhub-kubespawner.readthedocs.io/>, 2026. Accessed: Sep. 2026.
- [9] JupyterHub Development Team. Jupyterhub: A conceptual overview. <https://jupyterhub.readthedocs.io/en/latest/explanation/concepts.html>,

2026. Accessed: Sep. 2026.

- [10] JupyterHub Development Team. Spawners. <https://jupyterhub.readthedocs.io/en/stable/reference/spawners.html>, 2026. Accessed: Sep. 2026.
- [11] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6769–6781, 2020.
- [12] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.
- [13] Kubernetes Authors. Kubernetes scheduler. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, 2026. Accessed: Sep. 2026.
- [14] Kubernetes Authors. Pod quality of service classes. <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>, 2026. Accessed: Sep. 2026.
- [15] Kubernetes Authors. Resource management for pods and containers. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>, 2026. Accessed: Sep. 2026.
- [16] Kubernetes Authors. Resource quotas. <https://kubernetes.io/docs/concepts/policy/resource-quotas/>, 2026. Accessed: Sep. 2026.
- [17] Aris Leivadeas and Matthias Falkner. A survey on intent-based networking. *IEEE Communications Surveys & Tutorials*, 25(1):625–655, 2023.
- [18] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.

- [19] Open Container Initiative. OCI image format specification. <https://github.com/opencontainers/image-spec>, 2026. Accessed: Sep. 2026.
- [20] Project Jupyter. JupyterHub documentation. <https://jupyterhub.readthedocs.io/>, 2026. Accessed: Sep. 2026.
- [21] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*, pages 3982–3992, 2019.
- [22] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [23] Qin Wu, Alexander Clemm, Mohamed Boucadair, Christian Jacquenet, and Chongfeng Xie. Intent-driven cloud resource design. In *Proceedings of the 2021 IEEE Cloud Summit*, pages 141–146, 2021.
- [24] Zero to JupyterHub Contributors. Zero to JupyterHub with Kubernetes. <https://z2jh.jupyter.org/>, 2026. Accessed: Sep. 2026.